



UNIVERSITATEA DIN BUCUREȘTI

**FACULTATEA DE
MATEMATICĂ ȘI INFORMATICĂ**



**SPECIALIZAREA BAZE DE DATE ȘI
TEHNOLOGII SOFTWARE**

KINETOCARE – PLATFORMĂ DISTRIBUITĂ PENTRU MANAGEMENTUL PARCURSULUI TERAPEUTIC

COORDONATOR ȘTIINȚIFIC
Conf. univ. dr. Radu Boriga

ABSOLVENT
Anelis-Ramona Niță-Gheorghiaș

BUCUREȘTI

2026

REZUMAT

Clinicile de recuperare medicală funcționează pe baza unor protocoale iterative, care includ evaluarea inițială, tratamentul și reevaluarea periodică a pacientului. Analiza soluțiilor informatice existente indică o limitare comună: tipul serviciului medical este selectat manual la programare, ceea ce poate genera erori operaționale și reduce trasabilitatea deciziilor clinice.

Prezenta lucrare propune KinetoCare, o platformă distribuită de management clinic care remediază aceste deficiențe la nivel arhitectural. Pe lângă gestionarea dosarelor clinice, a jurnalelor de simptome și facilitarea comunicării în timp real, contribuția algoritmică centrală constă în implementarea unui automat finit determinist în nucleul tranzacțional. Acesta determină automat tipul de serviciu corespunzător din istoricul clinic persistent al pacientului, eliminând selecția manuală și prevenind alocările eronate.

Arhitectura integrează șapte microservicii de domeniu cu baze de date dedicate, un API Gateway cu agregare reactivă și mesagerie asincronă prin RabbitMQ. Lucrarea include, de asemenea, un algoritm iterativ cu fereastră glisantă pentru calculul disponibilității orare, un model de securitate Zero-Trust extins pe canalele persistente WebSocket/STOMP și un mecanism de consistență prin tranzacții compensatorii Saga pentru scenariile de scriere duală între furnizorul de identitate extern și baza de date locală.

Validarea sistemului demonstrează prevenirea rezervărilor duble prin blocare pesimistă la nivel de bază de date, confirmată prin teste de integrare cu 50 de fire de execuție simultane, alături de verificarea funcțională a securității STOMP și a mecanismului compensatoriu Saga. Rezultatele obținute demonstrează fezabilitatea utilizării arhitecturilor de microservicii în aplicații *e-Health* de nișă și relevanța automatizării deciziilor operaționale în procese medicale repetitive. KinetoCare oferă un model replicabil pentru domenii medicale bazate pe protocoale iterative de îngrijire, contribuind la transformarea aplicațiilor de programare din instrumente administrative pasive în sisteme de suport clinic activ.

ABSTRACT

Medical rehabilitation clinics operate according to iterative clinical protocols that include initial evaluation, treatment, and periodic patient re-evaluation. The analysis of existing software solutions indicates a common limitation: the medical service type is manually selected during appointment scheduling, which may lead to operational errors and reduce the traceability of clinical decisions.

This thesis proposes KinetoCare, a distributed clinical management platform that addresses these deficiencies at the architectural level. Alongside managing clinical records, symptom journals, and facilitating real-time communication, the central algorithmic contribution consists of implementing a deterministic finite automaton within the transactional core. This automatically derives the applicable service type from the patient's persistent clinical history, eliminating manual selection and preventing erroneous procedure assignments.

The architecture integrates seven domain microservices with dedicated databases, an API Gateway with reactive aggregation, and asynchronous messaging through RabbitMQ. The thesis also includes an iterative sliding-window algorithm for computing time-slot availability, a Zero-Trust security model extended to persistent WebSocket/STOMP channels, and a consistency mechanism based on Saga-style compensating transactions for dual-write scenarios between the external identity provider and the local database.

System validation demonstrates the prevention of double-booking through database-level pessimistic locking, confirmed by integration tests with 50 concurrent execution threads, alongside the functional verification of STOMP security and the Saga compensatory mechanism. The obtained results demonstrate the feasibility of using microservice architectures in niche e-Health applications and the relevance of automating operational decisions in repetitive medical processes. KinetoCare provides a replicable model for medical domains based on iterative care protocols, contributing to the transformation of scheduling applications from passive administrative tools into active clinical support systems.

CUPRINS

CAPITOLUL 1. INTRODUCERE	6
CAPITOLUL 2. PRELIMINARII ȘI ANALIZA DOMENIULUI	8
2.1 NOȚIUNI FUNDAMENTALE TEHNOLOGICE	8
2.2 PROVOCĂRILE DIN MANAGEMENTUL CLINIC	9
2.3 ANALIZA SOLUȚIILOR EXISTENTE ȘI POZIȚIONAREA SISTEMULUI	10
2.4 CERINȚELE SISTEMULUI	11
2.4.1 <i>Cerințe funcționale</i>	11
2.4.2 <i>Cerințe non-funcționale</i>	12
CAPITOLUL 3. ALEGEREA TEHNOLOGIILOR ȘI A STIVEI SOFTWARE	13
3.1 JUSTIFICAREA ARHITECTURII BAZATE PE MICROSERVICII	13
3.2 PRINCIPIUL CAP ȘI COMPROMISURI ARHITECTURALE ASUMATE	13
3.3 ECOSISTEMUL BACKEND: SPRING, WEBFLUX, FEIGN ȘI RABBITMQ	14
3.4 INTERFAȚA UTILIZATOR ȘI COMUNICAREA ÎN TIMP REAL	14
3.5 SECURITATE: KEYCLOAK, JWT ȘI MODELUL ZERO-TRUST	15
3.6 INFRASTRUCTURĂ ȘI PERSISTENȚĂ	15
CAPITOLUL 4. ARHITECTURA SISTEMULUI	16
4.1 HARTA MICROSERVICIILOR ȘI DELIMITAREA CONTEXTELOR (BOUNDED CONTEXTS)	16
4.2 FLUXUL DE COMUNICARE INTER-SERVICII	18
4.3 API GATEWAY ȘI TIPARUL BACKEND-FOR-FRONTEND (BFF)	19
4.4 MODELUL DE SECURITATE ZERO-TRUST DISTRIBUIT	19
4.5 IZOLAREA DATELOR ȘI MODELUL PERSISTENȚEI (DATABASE-PER-SERVICE)	21
4.5.1 <i>Fundamentarea modelului relațional și nivelul de izolare</i>	21
4.5.2 <i>Structura schemelor și normalizarea datelor</i>	22
4.5.3 <i>Strategia de indexare</i>	23
4.6 REZILIENȚA APLICATIVĂ ȘI TOLERANȚA LA DEFECTE	23
4.7 TOPOLOGIA ASINCRONĂ A EVENIMENTELOR DE NOTIFICARE	24
CAPITOLUL 5. IMPLEMENTĂRI TEHNICE ȘI LOGICA DE SISTEM	24
5.1 GESTIUNEA TRAIECTORIEI CLINICE PE BAZA STĂRII DATELOR	24
5.2 GENERAREA FERESTRELOR DE DISPONIBILITATE PRIN PROCESARE ITERATIVĂ	25
5.3 PARTIȚIONAREA TEMPORALĂ ȘI ANOMALIILE INTEROGĂRILOR LA MIEZUL NOPTII	26
5.4 OPTIMIZAREA AGREGĂRII DATELOR ȘI REZOLVAREA PROBLEMEI N+1	26
5.5 CONSISTENȚA ASIMETRICĂ LA DEZACTIVAREA CONTURILOR	27

5.6 IDEMPOTENȚA CONSUMATORILOR DE MESAJE	27
CAPITOLUL 6. INTERFAȚA CU UTILIZATORUL ȘI FLUXURILE OPERAȚIONALE	27
6.1 FLUXUL PACIENTULUI: DE LA PROFILARE CLINICĂ LA MONITORIZARE	28
6.2 FLUXUL OPERAȚIONAL AL TERAPEUTULUI: CALENDAR ȘI DOCUMENTARE	29
6.3 PANOUL DE ADMINISTRARE ȘI AGREGAREA DATELOR STATISTICE	30
6.4 COMUNICAREA TRANSVERSALĂ: NOTIFICĂRI ȘI MESAGERIE ÎN TIMP REAL	31
CAPITOLUL 7. VALIDARE, TESTARE ȘI PERFORMANȚĂ	32
7.1 TESTAREA UNITARĂ A COMPONENTELOR ALGORITMICE CRITICE	32
7.2 TESTAREA DE INTEGRARE A CONCURENȚEI TRANZACȚIONALE	33
7.3 VALIDAREA FUNCȚIONALĂ A INFRASTRUCTURII DISTRIBUITE	33
7.4 VALIDAREA REZILIENȚEI ÎN INTERFAȚA UTILIZATORULUI	34
7.5 VALIDAREA REZILIENȚEI ÎN INFRASTRUCTURA BAZATĂ PE CONTAINERE	35
CAPITOLUL 8. LIMITĂRI ARHITECTURALE ȘI DIRECȚII VIITOARE	35
8.1 LIMITĂRI ARHITECTURALE ȘI TEHNICE ASUMATE	35
8.2 DIRECȚII VIITOARE DE DEZVOLTARE ȘI SCALARE	37
CAPITOLUL 9. CONCLUZII	37
BIBLIOGRAFIE	39
ANEXE	41
ANEXA 1. TABEL COMPARATIV AL FUNCȚIONALITĂȚILOR SOLUȚIILOR SOFTWARE EXISTENTE	41
ANEXA 2. REGISTRUL MICROSERVICIILOR PLATFORMEI KINETOCARE	42
ANEXA 3. MATRICEA DE COMUNICARE INTER-SERVICII (SINCRON ȘI ASINCRON)	43
ANEXA 4. ARHITECTURA SERVICIILOR ȘI A FLUXURILOR DE REȚEA	45
ANEXA 5. ARHITECTURA ȘI MODELAREA BAZELOR DE DATE	46
ANEXA 6. DIAGrame TEHNICE ȘI LOGICA DE SISTEM	47
ANEXA 7. FLUXURILE OPERAȚIONALE ȘI INTERACȚIUNEA DIN INTERFAȚA UTILIZATORULUI	51
ANEXA 8. VALIDAREA SISTEMULUI, TESTAREA PERFORMANȚEI ȘI REZILIENȚA INFRASTRUCTURII	55

Capitolul 1. Introducere

Lucrarea de față este o disertație cu caracter aplicativ, care se încadrează în subdomeniul sistemelor software distribuite și al ingineriei bazelor de date relaționale, cu aplicabilitate în informatica medicală (*healthcare informatics*). Proiectul constă în proiectarea și implementarea de la zero a platformei KinetoCare, acoperind întregul ciclu de dezvoltare — de la analiza cerințelor până la *deployment-ul* containerizat în Kubernetes.

KinetoCare este un sistem distribuit destinat cabinetelor de kinetoterapie și recuperare medicală, care gestionează parcursul terapeutic al pacientului: înregistrarea și autentificarea securizată, crearea dosarului medical, căutarea și selectarea terapeutului, rezervarea inteligentă a ședințelor, comunicarea în timp real și colectarea feedback-ului subiectiv post-ședință (niveluri de durere, oboseală, dificultate). Din punct de vedere arhitectural, sistemul este alcătuit din șapte microservicii de domeniu, fiecare cu propria bază de date MySQL izolată, și un API Gateway *stateless*, conform tiparului *database-per-service*. Comunicarea internă combină apeluri REST sincrone pentru operațiunile tranzacționale cu evenimente asincrone prin RabbitMQ pentru notificări și alerte; modulul de mesagerie pacient-terapeut operează prin canale WebSocket/STOMP persistente. Interfața client este o aplicație React (SPA).

Motivația alegerii acestei teme a pornit de la observația că soluțiile naționale și internaționale actuale sunt solide pentru documentație și facturare, dar nu gestionează automat parcursul terapeutic și nu integrează o comunicare în timp real, lăsând loc pentru erori umane în deciziile operaționale. Scopul lucrării a fost proiectarea și implementarea unei soluții care să acopere aceste lacune: un sistem capabil să elimine selecția manuală a procedurilor prin determinarea automată a tipului de serviciu direct la rezervare, pe baza istoricului persistent al pacientului.

Contribuția proprie acoperă întregul ciclu de dezvoltare software: analiza pieței și formalizarea cerințelor funcționale și non-funcționale, proiectarea arhitecturală a celor opt componente, implementarea completă a backend-ului și frontend-ului, și validarea prin teste unitare și de integrare. Dintre soluțiile tehnice elaborate, se remarcă: un automat finit determinist care calculează tipul de serviciu clinic aplicabil pe baza istoricului medical al pacientului; un algoritm iterativ cu fereastră glisantă pentru generarea sloturilor de disponibilitate orară cu *buffere* operaționale; mecanisme de securitate Zero-Trust extinse la nivelul canalelor WebSocket/STOMP prin interceptori dedicați; și asigurarea consistenței distribuite între baza

de date locală și furnizorul de identitate Keycloak prin tranzacții compensatorii (tipar Saga simplificat).

Lucrarea este structurată în nouă capitole:

- **Capitolul 1** prezintă contextul general al platformei, motivația dezvoltării, contribuțiile proprii și modul de structurare a documentului.
- **Capitolul 2** introduce cadrul teoretic și tehnologic al platformei, analizează limitele soluțiilor concurente printr-o matrice comparativă și stabilește cerințele funcționale și non-funcționale ale sistemului.
- **Capitolul 3** justifică tranziția de la modelul monolitic la arhitectura de microservicii, argumentează compromisurile arhitecturale asumate și fundamentează selecția stivei tehnologice (Spring Boot, React, RabbitMQ, WebSocket, Docker, Kubernetes).
- **Capitolul 4** detaliază topologia distribuită a sistemului — delimitarea celor șapte contexte de domeniu, modelul de securitate Zero-Trust cu validare descentralizată a jetoanelor JWT și strategia de persistență a datelor cu indecși adaptați tiparelor de interogare.
- **Capitolul 5** documentează mecanismele decizionale centrale: automatul finit clinic, algoritmul de disponibilitate cu fereastră glisantă, tratarea anomaliei de interogare la granița calendaristică a zilei, optimizarea agregării datelor inter-servicii prin procesare în lot, consistența asimetrică în operațiunile de dezactivare a conturilor și garantarea idempotenței consumatorilor de mesaje.
- **Capitolul 6** descrie modul în care arhitectura se reflectă în interfața grafică, detaliind fluxurile operaționale ale celor trei roluri, optimizările implementate în stratul de prezentare, precum și modulele transversale de notificări și mesagerie în timp real.
- **Capitolul 7** prezintă strategia de validare adoptată: testarea algoritmilor în izolare, testele de stres tranzacțional la concurență ridicată, verificarea comportamentului infrastructurii containerizate în scenarii de defecțiune parțială și validarea mecanismelor de reziliență la nivelul interfeței utilizatorului.
- **Capitolul 8** documentează compromisurile tehnice asumate în contextul unui produs minim viabil (MVP) și propune direcțiile arhitecturale de evoluție.
- **Capitolul 9** sintetizează rezultatele obținute și relevanța platformei în contextul digitalizării serviciilor medicale din România.

Capitolul 2. Preliminarii și analiza domeniului

Acest capitol prezintă cadrul teoretic și tehnologic necesar pentru înțelegerea platformei KinetoCare. Sunt abordate conceptele de bază ale domeniului, precum și justificarea necesității sistemului și a ariei sale de acoperire. De asemenea, sunt definite cerințele funcționale și non-funcționale care stau la baza deciziilor de proiectare arhitecturală din capitolele următoare.

2.1 Noțiuni fundamentale tehnologice

Secțiunea introduce conceptele teoretice pe care se sprijină arhitectura platformei: sistemele distribuite și principiile *domain-driven design*, modelele de persistență relațională și comunicare asincronă prin AMQP, arhitectura SPA a interfeței client, mecanismele de securitate Zero-Trust bazate pe JWT și Keycloak și infrastructura de containerizare Docker și Kubernetes.

Modelul sistemelor distribuite și ecosistemul backend. Fundamentul teoretic al aplicației constă în descompunerea complexității domeniului medical prin arhitecturi bazate pe microservicii [2]. Această abordare arhitecturală a fost implementată aplicând principiile *domain-driven design* (DDD), care definesc granițe logice stricte (*bounded contexts*) pentru fiecare componentă de business în parte [8]. Infrastructura de procesare utilizează mediul de execuție concurent al Mașinii Virtuale Java (JVM) [6], orchestrat prin ecosistemul modular Spring. Abstracțiile oferite de Spring Boot și Spring Cloud facilitează implementarea tiparelor specifice sistemelor distribuite: auto-configurarea, rutarea la marginea rețelei prin API Gateway, comunicarea declarativă inter-servicii prin OpenFeign și optimizarea performanței prin stocarea temporară a rezultatelor în memorie prin Spring Cache [7].

Persistența datelor și modele de comunicare. Garantarea consistenței datelor clinice impune utilizarea unor sisteme de persistență care respectă strict proprietățile tranzacționale ACID. Utilizarea sistemelor de gestiune a bazelor de date relaționale (MySQL InnoDB) permite controlul concurenței la nivel de rând (*row-level locking*) și optimizarea căutărilor prin structuri de tip index grupat (Clustered Index), mecanisme necesare pentru izolarea tranzacțiilor la nivelul fiecărui microserviciu [17].

Comunicarea în cadrul sistemului este hibridă, adaptată constrângerilor de timp. Pentru decuplarea temporală a serviciilor auxiliare (precum alertele și notificările), se utilizează un model orientat pe evenimente, implementat prin protocolul deschis AMQP și brokerul

RabbitMQ, asigurând rutarea semantică și izolarea mesajelor eronate în cozi de carantină (*dead letter queues*) [9]. Separat, modulul de mesagerie instantanee operează prin canale bidirecționale persistente (WebSocket) [14], iar la nivel aplicativ, rutarea cadrelor de mesaje este gestionată prin protocolul STOMP [15], utilizând SockJS ca mecanism de fallback pentru mediile de rețea care restricționează conexiunile TCP directe [16].

Arhitectura frontend și interfața utilizator. Arhitectura interfeței client urmează modelul de tip Single Page Application (SPA), mutând responsabilitatea rutării și a randării de pe server direct în browserul utilizatorului. Această arhitectură este materializată prin biblioteca React, care utilizează un model eficient de actualizare a interfeței bazat pe arborele virtual al documentului (Virtual DOM) [10]. Ecosistemul este completat de instrumente de rutare (React Router) [11] și biblioteci specializate pentru redarea graficelor vectoriale statistice (Recharts) [12] și managementul agendei (FullCalendar) [13], reducând masiv volumul de date transferat prin rețea la fiecare interacțiune.

Securitate zero-trust și infrastructură imutabilă. Securizarea unui sistem distribuit necesită abandonarea modelului perimetral de încredere. Platforma delegă gestiunea identității și a politicilor de acces platformei Keycloak [19], bazată pe standardul deschis de autorizare OAuth 2.0 [18]. Propagarea identității între microserviciile interne se realizează criptografic, prin jetoane compacte de tip JWT (JSON Web Token) [5], permițând fiecărui serviciu să valideze descentralizat drepturile de acces.

La nivel operațional, izolarea execuției și garantarea parității între mediile de dezvoltare și producție se bazează pe conceptul de containerizare (Docker) [20]. Disponibilitatea sistemului este gestionată algoritmic printr-o platformă de orchestrare (Kubernetes), care utilizează mecanisme de monitorizare a stării (*readiness și liveness probes*) pentru a direcționa traficul valid și a recicla automat procesele nefuncționale [21].

2.2 Provocările din managementul clinic

Sectorul de recuperare medicală și kinetoterapie se distinge de alte ramuri medicale prin natura sa iterativă, implicând un traseu terapeutic de lungă durată, structurat pe multiple ședințe, evaluări și reevaluări periodice. În România, clinicile operează frecvent cu instrumente generice nespecializate, precum agendele fizice sau foile de calcul, context care generează provocări operaționale și tehnice majore [22].

În primul rând, utilizarea acestor metode rudimentare conduce la fragmentarea fluxului informațional, introducând riscuri ridicate de suprapunere a programărilor și ineficiențe severe în optimizarea orarului terapeuților. În al doilea rând, absența trasabilității clinice și calculul manual al raportului dintre ședințele efectuate și cele prescrise predispun la ratarea momentului optim pentru reevaluarea medicală a pacientului. Managementul resurselor este de asemenea deficitar, deoarece lipsa sincronizării în timp real a disponibilităților multi-locatie și a perioadelor de concediu limitează vizibilitatea operațională. Mai mult, sistemele tradiționale nu oferă o buclă digitală de feedback prin care pacientul să raporteze periodic nivelul de durere sau oboseală după fiecare ședință. În cele din urmă, utilizarea unor evidențe gestionate cu instrumente nespecializate introduce vulnerabilități critice de securitate, întrucât aceste sisteme nu oferă mecanisme avansate de control al accesului bazat pe roluri (RBAC) sau jurnale de audit imutabile (*audit trail*), ceea ce poate conduce la neconformități cu cerințele GDPR [1].

2.3 Analiza soluțiilor existente și poziționarea sistemului

Piața aplicațiilor destinate managementului recuperării medicale este divizată între soluții internaționale complexe și platforme locale adaptate contextului legislativ autohton. Soluțiile de tip Electronic Medical Record (EMR) internaționale, precum WebPT [23] sau Pabau [24], oferă instrumente solide pentru documentația clinică, facturare și administrarea proceselor de business multi-locatie, însă nu integrează module dedicate monitorizării feedback-ului subiectiv al pacientului, canale de mesagerie instantanee pacient-terapeut sau mecanisme automate de gestionare a planurilor terapeutice bazate pe bugete stricte de ședințe. Pe de altă parte, platformele românești specifice fizioterapiei, cum sunt FizioHub [25] și ReKi [26], facilitează prescrierea procedurilor și integrarea cu serviciile fiscale naționale (e-Factura), dar păstrează caracterul exclusiv manual al selecției serviciilor clinice la programare și omit implementarea unor canale de comunicare directă în timp real. O sinteză granulară a acestor criterii funcționale evaluate comparativ este structurată sub forma unei matrice în Anexa 1.

Sinteza lacunelor analizate evidențiază o lipsă sistematică de automatizare a desfășurării clinice a pacientului, decizia de alocare a serviciilor medicale în momentul rezervării rămânând exclusiv manuală și predispusă la erori umane. Platforma KinetoCare abordează acest deficit structural prin implementarea unui motor decizional care determină automat tipul de serviciu aplicabil (evaluare inițială, tratament standard sau reevaluare) direct pe baza istoricului medical salvat în persistența sistemului (mecanism detaliat în Secțiunea 5.1). Suplimentar, proiectul propus integrează un canal de mesagerie în timp real protejat prin validare clinică activă și un

modul prin care pacientul completează periodic un jurnal de simptome subiective. Ca limitare asumată în arhitectura curentă, versiunea actuală exclude modulele de facturare și decontare prin sistemul public de asigurări (CNAS), prioritizând dezvoltarea nucleului tranzacțional al sistemului și garantarea consistenței datelor medicale.

2.4 Cerințele sistemului

Specificațiile tehnice și funcționale ale platformei KinetoCare au fost derivate direct din analiza limitărilor pieței comerciale și din constrângerile specifice domeniului clinic. Necesitatea acestor cerințe este fundamentată de studiile recente privind maturitatea digitală a serviciilor medicale din România [22], care evidențiază că sistemul național de sănătate nu răspunde pe deplin nevoilor actuale ale pacienților din cauza fragmentării datelor și a lipsei unor canale digitale directe de interacțiune. Pentru a răspunde structural acestor provocări, specificațiile sunt împărțite în cerințe funcționale, ce definesc comportamentul așteptat al modulelor, și cerințe non-funcționale, care impun proprietățile de calitate și performanță asupra întregului sistem.

2.4.1 Cerințe funcționale

Cerințele funcționale ale platformei KinetoCare sunt structurate pe module operaționale determinate strict de rolurile utilizatorilor. Modulul de autentificare și securitate guvernează crearea conturilor diferențiate bazate pe roluri (pacient, terapeut, administrator), autentificarea securizată, gestionarea centralizată a sesiunilor active și asigurarea revocării complete a drepturilor de acces în momentul deconectării.

Pentru utilizatorii cu **rolul de pacient**, sistemul impune completarea obligatorie a datelor demografice și clinice minime înainte de deblocarea interfeței, permite căutarea terapeuților într-un catalog filtrabil, automatizează alocarea tipului de serviciu clinic la rezervarea sloturilor orare, previne suprapunerile concurente și oferă un formular pentru transmiterea feedback-ului subiectiv post-ședință.

Modulul dedicat terapeutului permite configurarea programului de lucru pe locații fizice și gestionarea concediilor, oferă vizualizarea agendei clinice zilnice și actualizarea stării ședințelor, facilitează documentarea diagnosticelor și a planurilor de recuperare, asigură accesul la dosarul clinic unificat al pacienților (istoricul clinic și evoluția simptomelor raportate) și transmite alerte automate în momentul în care un caz necesită reevaluare clinică.

La nivelul **modulului administrativ**, platforma oferă instrumente pentru gestiunea catalogului de servicii și locații, alături de suspendarea conturilor de utilizator cu propagarea anulărilor în lanț și accesarea unui tablou de raportare statistică pentru monitorizarea activității globale.

Modulele transversale completează specificațiile funcționale prin facilitarea comunicării bidirecționale instantanee între pacienți și terapeuți, validarea automată a dreptului de comunicare pe baza existenței unei relații clinice active și expedierea asincronă a notificărilor automate bazate pe evenimente, cum sunt confirmările de programare și reminderele președință.

2.4.2 Cerințe non-funcționale

Cerințele non-funcționale definesc limitele în care au fost luate deciziile de proiectare arhitecturală ale platformei KinetoCare. În ceea ce privește securitatea și protecția datelor, sistemul trebuie să valideze drepturile de acces pentru fiecare acțiune la nivel local, să asigure izolarea completă a datelor medicale sensibile și să protejeze sesiunile împotriva vulnerabilităților web standard. Scalabilitatea și performanța impun ca infrastructura să susțină creșterea volumului de utilizatori fără degradarea performanței, timpii de răspuns pentru operațiunile standard de interogare înscriindu-se în limitele percepute ca fluide de către client. Integritatea și consistența datelor presupun că nicio operațiune critică concurentă nu poate lăsa sistemul într-o stare invalidă; în caz de eroare, mecanismele de recuperare restaurează o stare validă înainte de a permite reluarea procesării.

Reziliența și toleranța la defecte asigură faptul că indisponibilitatea temporară a unui modul secundar nu afectează funcționalitatea nucleului clinic, sistemul respectând principiul degradării controlate a interfeței. Din punct de vedere legal, auditabilitatea este garantată prin jurnalizarea automată a oricărei modificări aduse entităților clinice și operaționale critice ale sistemului — date medicale (fișe, evaluări, programări, jurnale de simptome), servicii, conturi și profiluri de utilizator —, marcajele temporale și identificatorul operatorului (UUID-ul Keycloak) fiind stocate imutabil, asigurând trasabilitatea completă conform normelor GDPR [1]. În final, portabilitatea impune ca sistemul să poată fi implementat în medii de cloud diferite cu modificări minime de configurare, în timp ce uzabilitatea obligă ca interfața grafică să fie optimizată pentru dispozitive mobile, să afișeze erorile de validare în context și să prevină transmiterea de cereri duplicate prin dezactivarea controalelor interactive pe durata procesării HTTP.

Capitolul 3. Alegerea tehnologiilor și a stivei software

Acest capitol argumentează deciziile de selecție tehnologică și arhitecturală, demonstrând necesitatea tranziției de la modelul monolitic la un sistem distribuit și evaluând compromisurile arhitecturale impuse de această abordare.

3.1 Justificarea arhitecturii bazate pe microservicii

În faza de analiză, varianta monolitică a fost eliminată din cauza a trei incompatibilități structurale. Prima ține de profilurile de încărcare radical diferite: subsistemul de mesagerie menține conexiuni TCP persistente cu trafic continuu, în timp ce catalogul de servicii implică interogări rare și date stabile — o combinație greu de optimizat eficient într-un singur proces. A doua vizează regimurile de consistență: rezervările necesită confirmare imediată, pe când notificările tolerează o scurtă întârziere; gestionarea ambelor regimuri în același strat tranzacțional ar fi complicat inutil codul. A treia privește modelul de I/O: un strat reactiv la marginea sistemului amestecat cu componente blocante în același proces ar fi anulat avantajele programării reactive. Sistemul a fost proiectat, în consecință, ca o suită de servicii independente, organizate conform principiilor *domain-driven design* în granițe logice stricte (*bounded contexts*). Registrul complet al microserviciilor, cu porturile și responsabilitățile de domeniu asociate, este disponibil în Anexa 2.

3.2 Principiul CAP și compromisuri arhitecturale asumate

Conform principiului CAP (*Consistency, Availability, Partition Tolerance*) [27], arhitectura adoptă o abordare hibridă în scenariile de partiționare a rețelei. Operațiile de rezervare favorizează consistența în detrimentul disponibilității (CP), respingând cererile atunci când lanțul de apeluri sincrone inter-servicii este întrerupt sau când validarea descentralizată a jetoanelor de acces eșuează din cauza indisponibilității infrastructurii de identitate. În schimb, procesele auxiliare de notificare prioritizează disponibilitatea (AP), utilizând un model de consistență eventuală bazat pe RabbitMQ [9]. Separat de teorema CAP, problema suprapunerilor la rezervare, care ține de concurență, nu de partiție, este rezolvată la nivelul persistenței prin tranzacții ACID [17].

Această abordare introduce trei compromisuri arhitecturale. Problema scrierilor duale (Dual Write) este gestionată printr-o Saga cu tranzacții compensatorii [3], care declanșează ștergerea

contului creat în Keycloak în cazul eșecului persistenței locale. Imposibilitatea realizării JOIN-urilor între bazele de date ale serviciilor este abordată prin tiparul Snapshot [3], care păstrează o copie a valorilor tarifare existente la momentul rezervării, asigurând integritatea istoricului financiar. În final, complexitatea operațională asociată administrării celor opt servicii este redusă prin standardizarea stivei tehnologice și prin orchestrarea containerizată a componentelor sistemului.

3.3 Ecosistemul backend: Spring, WebFlux, Feign și RabbitMQ

Infrastructura backend este construită pe ecosistemul Spring Boot, cu gestionarea centralizată a dependențelor prin intermediul unui proiect multi-modul Maven. În funcție de caracteristicile fiecărei componente, sunt utilizate atât modele reactive, cât și modele imperative. API Gateway-ul reprezintă singura componentă bazată pe Spring WebFlux, alegere care facilitează implementarea tiparului Backend-for-Frontend (BFF) [2], descris în detaliu în Secțiunea 4.3.

Pentru comunicațiile sincrone dintre servicii sunt utilizați clienți declarativi Spring Cloud OpenFeign. Această tehnologie simplifică integrarea prin generarea automată a implementărilor de clienți HTTP pe baza interfețelor adnotate. Pentru a asigura stabilitatea apelurilor sincrone, platforma impune o structură în care serviciile tranzacționale (precum *programari-service*) apelează servicii frunză (de tipul *terapeuti-service*), reducând riscul de dependențe circulare. Acolo unde interogările bidirecționale sunt necesare funcțional (de exemplu, între *programari-service*, *pacienti-service* și *user-service*), acestea sunt decuplate la nivel de execuție prin tratarea defensivă a excepțiilor de rețea și prin delegarea fluxurilor de notificare și alertare pe canalul de mesagerie asincronă.

Comunicarea asincronă este realizată prin RabbitMQ, ales în locul Apache Kafka datorită suportului nativ pentru rutarea mesajelor prin *topic exchange* și a mecanismelor dedicate de izolare a mesajelor eronate prin *dead letter queues* (DLQ) [9].

3.4 Interfața utilizator și comunicarea în timp real

Aplicația client este implementată ca o Single Page Application (SPA), utilizând React și React Router [10, 11] pentru a evita reîncărcările complete de pagină, navigarea bazându-se exclusiv pe interogări HTTP de date în format JSON către API Gateway. React a fost selectat datorită flexibilității ecosistemului și posibilității de integrare modulară a unor biblioteci specializate,

precum FullCalendar și Recharts. La nivel arhitectural, codul este organizat pe patru straturi logice: rutare, prezentare, integrare cu backend-ul și gestionarea stării globale.

Subsistemul de mesagerie în timp real utilizează protocolul WebSocket pentru a asigura o comunicare bidirecțională peste o conexiune TCP persistentă, reducând supraîncărcarea de rețea (*overhead*) asociată cererilor HTTP clasice [14]. La nivel aplicativ, rutarea mesajelor și gestionarea abonărilor la canalele de comunicație se realizează prin protocolul STOMP [15], utilizând SockJS ca mecanism de rezervă (*fallback*) pentru mediile de rețea restrictive [16]. Deoarece conexiunea TCP ocolește filtrele HTTP standard din Gateway, securitatea acestui canal a necesitat o soluție dedicată, detaliată în Secțiunea 4.4.

3.5 Securitate: Keycloak, JWT și modelul Zero-Trust

Gestionarea identității este delegată platformei Keycloak pentru managementul accesului și al identității (IAM), evitându-se astfel riscurile asociate unei implementări proprii a mecanismelor de autentificare. Sistemul adoptă modelul de securitate Zero-Trust [4], ceea ce înseamnă că rețeaua internă nu este considerată implicit sigură, iar fiecare microserviciu își validează local și criptografic jetoanele JWT [5] primite. Detaliile privind separarea tokenurilor la nivelul API Gateway-ului și validarea distribuită a identității sunt prezentate în Secțiunile 4.3 și 4.4.

3.6 Infrastructură și persistență

Izolarea logică stabilită în faza de dezvoltare este transpusă la nivel de infrastructură prin containerizarea cu Docker [20]. Pentru aplicația frontend, s-au folosit construcții de tip multi-stage cu scopul de a livra doar fișierele statice optimizate, prin intermediul unui server Nginx, reducând astfel dimensiunea suprafeței de atac. Orchestrarea containerelor în mediul de producție este gestionată cu ajutorul Kubernetes [21], care asigură reziliența sistemului prin mecanisme de tip *liveness* și *readiness*. Aceste mecanisme previn direcționarea traficului către instanțe aflate în faza de inițializare și permit repornirea automată a containerelor care nu mai răspund la verificările de sănătate. Această abordare, combinată cu separarea prin spații de nume (*namespaces*) și politici stricte de rețea, aplică practic principiul apărării stratificate (*defense in depth*).

Pentru stocarea datelor din cele șapte scheme izolate, s-a ales motorul MySQL 8 InnoDB. Decizia a fost fundamentată pe avantajele structurii de index grupat (*clustered index*), organizat

ca un arbore B+Tree direct pe cheia primară, configurație care accelerează căutările după identificator unic — un tipar de interogare frecvent în arhitecturile de microservicii. Detaliile privind nivelul de izolare al tranzacțiilor și strategiile de blocare pesimistă utilizate pentru prevenirea blocajelor de tip *deadlock* sunt prezentate în Secțiunea 4.5.1.

Capitolul 4. Arhitectura Sistemului

Acest capitol detaliază topologia distribuită a platformei KinetoCare, delimitarea contextelor logice de business și mecanismele de garantare a securității și consistenței datelor într-un ecosistem bazat pe microservicii.

4.1 Harta microserviciilor și delimitarea contextelor (bounded contexts)

Arhitectura platformei KinetoCare aplică principiile *domain-driven design* (DDD) și organizează sistemul pe patru niveluri de execuție cu responsabilități strict delimitate.

Nivelul client (*client layer*) cuprinde aplicația React SPA [10], singurul canal de interacțiune vizuală pentru toți utilizatorii platformei.

Nivelul de margine (*edge layer*) este format din API Gateway și serverul de identitate Keycloak [19], care împreună constituie perimetrul de securitate față de exterior: Gateway-ul guvernează rutarea, agregarea datelor și izolarea jetoanelor de sesiune, iar Keycloak gestionează exclusiv ciclurile de autentificare.

Nivelul serviciilor de domeniu (*domain services layer*) cuprinde șapte microservicii backend, fiecare responsabil de un singur context delimitat logic, cu model propriu de date și persistență independentă.

Nivelul de date (*data layer*) este alcătuit din șapte baze de date relaționale MySQL izolate [17], fiecare deținută exclusiv de un singur microserviciu; accesul direct al unui serviciu la schema altuia este interzis structural, orice necesitate de date inter-domenii rezolvându-se exclusiv prin API-urile expuse de fiecare serviciu. Topologia pe cele patru niveluri este ilustrată în Figura A4.1 din Anexa 4.

Cele șapte microservicii de domeniu corespund fiecare unui bounded context distinct, cu model de date propriu și graniță de responsabilitate strict delimitată.

Contextul de identitate și autentificare (user-service) funcționează ca strat de abstractizare peste furnizorul de identitate Keycloak, menținând o oglindă optimizată a datelor de identitate în baza de date locală pentru a elimina dependența sincronă de Keycloak la fiecare cerere internă. Responsabilitatea critică a acestui context este orchestrarea cascadei de dezactivare: la suspendarea unui cont, starea este propagată prin apeluri sincrone către profilurile medicale și sistemul de programări, invalidând toate operațiunile viitoare.

Contextul profil clinic (pacienti-service) extinde identitatea de bază cu attributele specifice istoricului medical — date demografice sensibile și preferințe clinice — și gestionează jurnalul subiectiv de progres, prin care feedback-ul pacientului (niveluri de durere, oboseală și dificultate) este capturat după fiecare ședință și vizualizat ca grafic evolutiv în dosarul clinic al terapeutului.

Contextul resurse umane și disponibilitate (terapeuti-service) modelează trei subdomenii interconectate: profilul profesional, infrastructura fizică (catalogul de locații) și managementul timpului (ferestre de disponibilitate săptămânală și perioade de concediu). Din perspectivă topologică, funcționează ca nod frunză: nu inițiază niciun apel de rețea către alte componente și este exclusiv interogat de celelalte servicii pentru validarea conflictelor temporale și a perioadelor de indisponibilitate.

Contextul nucleu tranzacțional clinic (programari-service) co-localizează patru entități cu cerințe stricte de consistență locală: programare, evaluare, evoluție și relație terapeutică. Această grupare este deliberată — o evaluare medicală este dependentă logic și tranzacțional de programarea care a declanșat-o; separarea lor ar fi impus protocoale costisitoare de coordonare distribuită. Tot în cadrul acestui context se aplică tiparul Snapshot detaliat în Secțiunea 4.5.2.

Contextul catalog de servicii (servicii-service) gestionează clasificarea intervențiilor medicale și politica tarifară a clinicii. Asemeni *terapeuti-service*, funcționează ca nod-frunză: nu inițiază apeluri de rețea și este exclusiv interogat de *programari-service* pentru extragerea atributelor comerciale la momentul rezervării.

Contextul mesagerie clinică (chat-service) combină o interfață REST pentru persistența arhivelor cu un flux WebSocket/STOMP [14, 15] pentru comunicarea instantanee. O caracteristică de siguranță arhitecturală este condiționarea transmiterii mesajelor de

confirmarea sincronă a existenței unei relații terapeutice active — un pacient nu poate iniția sau continua comunicarea cu un terapeut cu care relația clinică a fost încheiată.

Serviciul de alerte și notificări (notificari-service) funcționează ca un utilitar transversal de infrastructură, neavând un context de business de sine stătător în viziunea *domain-driven design*, ci acționând ca un simplu consumator tehnic. Acesta este orientat exclusiv pe evenimente (*event-driven*) și decuplat de tranzacțiile clinice de bază: consumă evenimente din infrastructura de mesagerie AMQP [9] și persistă notificările operaționale, fără a afecta disponibilitatea celorlalte module.

Deoarece tiparul *database-per-service* interzice interogările SQL de tip JOIN între scheme diferite, platforma adoptă o convenție globală de referențiere: identificatorul *keycloakId* (UUID), generat și garantat imutabil de furnizorul de identitate. Acesta servește ca cheie externă logică — fără constrângeri fizice la nivelul motorului SQL — în toate bazele de date ale platformei. Alegerea unui identificator extern față de cheile auto-incrementale locale elimină o dependență arhitecturală specifică sistemelor monolitice: un *id* local nu posedă semnificație în afara propriului microserviciu și nu poate servi ca referință stabilă la scara unui sistem distribuit.

4.2 Fluxul de comunicare inter-servicii

Alegerea stilului de comunicare este determinată de logica operațională și de sincronizarea tranzacțiilor.

Comunicarea sincronă (HTTP/REST) este implementată prin clienți declarativi OpenFeign [7] și utilizată pentru dependențele tranzacționale imediate. O excepție o reprezintă fluxul de înregistrare a utilizatorilor: în absența unui jeton JWT emis, inițializarea profilurilor în microserviciile vizate se realizează printr-un client simplu *RestTemplate*, fără propagarea contextului de securitate — altfel, Feign ar fi returnat *401 Unauthorized*, deoarece tokenul nu există încă. Deși topologia apelurilor sincrone include dependențe bidirecționale între anumite servicii, riscul de blocaj distribuit este atenuat prin gestionarea izolată a excepțiilor și prin migrarea fluxurilor de notificare și alertare pe canalul asincron RabbitMQ. Structura completă este detaliată în Tabelul A3.1 din Anexa 3.

Comunicarea asincronă (AMQP) este rezervată proceselor cu toleranță la consistență eventuală, precum notificările și alertele. Infrastructura utilizează RabbitMQ configurat cu

rutare semantică prin Topic Exchange [9]. Pentru a preveni blocarea fluxului principal în cazul mesajelor malformate, sistemul utilizează o rețea de carantinare, mecanism de reziliență detaliat în Secțiunea 4.6. Structura evenimentelor AMQP este detaliată în Tabelul A3.2 din Anexa 3.

Graful orientat al fluxurilor de comunicare inter-servicii este prezentat în Figura A4.2 din Anexa 4.

4.3 API Gateway și tiparul Backend-for-Frontend (BFF)

API Gateway-ul îndeplinește două roluri principale în arhitectura sistemului. Ca proxy invers și barieră de securitate, normalizează traficul, gestionează politicile CORS și rezolvă o problemă concretă de securitate a sesiunilor: după autentificarea prin Keycloak, Gateway-ul separă cele două jetoane — cel de reîmprospătare este stocat într-un *cookie HttpOnly*, inaccesibil codului JavaScript, iar cel de acces este expus browserului cu o durată scurtă de viață. În cazul unei vulnerabilități de tip XSS, compromiterea aplicației client nu permite preluarea persistentă a sesiunii utilizatorului, deoarece jetonul de reîmprospătare rămâne inaccesibil codului executat în browser. Al doilea rol este de orchestrator Backend-for-Frontend (BFF) [2]: în loc să oblige clientul să facă mai multe cereri separate pentru a asambla un ecran, Gateway-ul le lansează concurrent intern, procesează identificadorii în lot și returnează un răspuns unificat. Stiva reactivă WebFlux permite gestionarea unui număr mare de cereri simultane fără a bloca fire de execuție. Dacă un microserviciu din rețeaua internă returnează eroare, fluxul reactiv absoarbe excepția și construiește un răspuns parțial — interfața rămâne funcțională chiar și când o componentă secundară este indisponibilă.

O decizie deliberată a fost menținerea agregării dosarului clinic extins în *programari-service*, nu în Gateway. Mutarea acestei logici în Gateway ar fi transformat interogarea datelor clinice locale într-un set fragmentat de apeluri de rețea către microservicii diferite, anulând avantajele grupării lor tranzacționale și relaționale.

4.4 Modelul de securitate Zero-Trust distribuit

Arhitectura de securitate a platformei respinge modelul perimetral tradițional, conform căruia o cerere ce a traversat punctul de intrare este implicit considerată de încredere [4]. Fiecare microserviciu acționează ca un server de resurse autonom, validând independent orice jeton JWT [5] recepționat, indiferent de originea sa.

Pentru a evita transformarea serverului Keycloak într-un blocaj de performanță prin interogări sincrone la fiecare cerere, validarea identității este realizată descentralizat. Fiecare componentă de domeniu preia în mod leneș (*lazy*), la prima solicitare securizată, setul de chei publice prin *endpoint-ul* JWKS (*JSON Web Key Set*) expus de furnizorul de identitate (*identity provider*) și le menține în *cache* local. Semnătura asimetrică a jetonului (algoritmul RS256) este verificată local, fără comunicare sincronă cu Keycloak pentru fiecare cerere, iar rolurile extrase din structura JWT sunt traduse în autorități (*GrantedAuthority*) evaluate local.

Controlul accesului bazat pe roluri (RBAC) este distribuit pe două straturi complementare și redundante. La nivelul stratului de margine, API Gateway-ul aplică o politică generală de filtrare, blocând traficul neautorizat și restricționând direct rutele administrative specifice catalogului de locații (*hasRole("admin")*). Pentru celelalte resurse administrative, Gateway-ul doar impune ca cererea să fie autentificată, delegând controlul granular de acces către microserviciul de destinație. Odată traversat Gateway-ul, fiecare microserviciu aplică propriile reguli de acces prin configurări programatice ale lanțului de filtre Spring Security (*requestMatchers* asociate cu rolurile traduse din JWT) și validări imperative în stratul aplicativ (cum ar fi verificarea identității din contextul de securitate în raport cu proprietarul resurselor). Redundanța dintre cele două straturi este intenționată: o eventuală eroare de configurare a regulilor de rutare din Gateway nu compromite integritatea datelor medicale din profunzimea rețelei.

Modelul Zero-Trust a generat o provocare arhitecturală critică la nivelul modulului de mesagerie. Filtrele de securitate HTTP sunt concepute pentru ciclul cerere-răspuns: autentifică, procesează, resetează. Protocolul WebSocket debutează ca o cerere HTTP standard, dar după tranziția de protocol (HTTP 101 *Switching Protocols*), comunicarea continuă printr-un canal TCP persistent [14], ocolind complet lanțul de filtre HTTP. Consecința directă este că *SecurityContextHolder* rămâne gol pentru cadrele STOMP ulterioare, cauzând eșecul apelurilor OpenFeign subiacente cu eroarea 401 *Unauthorized*.

Soluția este interceptorul dedicat *StompSecurityInterceptor*, care restabilește lanțul de încredere în mod dinamic. În faza *preSend()*, la recepționarea unui cadru STOMP [15], interceptorul analizează tipul comenzii. Pentru cadrul inițial *CONNECT*, tokenul JWT este extras din antetul *Authorization* și decodat local pentru a asocia identitatea autentificată cu sesiunea WebSocket a utilizatorului (prin intermediul *accessor.setUser()*) și pentru a popula *SecurityContextHolder* pentru firul curent. Pentru cadrele ulterioare (precum *SEND*), deoarece

antetul *Authorization* lipsește, interceptorul restabilește automat identitatea în *SecurityContextHolder* utilizând obiectul de autentificare stocat în sesiune. În faza de execuție, logica de *business* rulează cu identitatea complet propagată, permițând clienților OpenFeign să transmită tokenul către microserviciile din aval. În final, faza *postSend()* curăță obligatoriu contextul apelând *SecurityContextHolder.clearContext()*, indiferent de rezultatul procesării. Deoarece serverele WebSocket reutilizează firele de execuție din *pool-uri*, această curățare este critică pentru a preveni *credential leakage* (menținerea credențialelor unui utilizator pe un fir utilizat ulterior de altcineva). Ciclul complet de viață al acestui interceptor pe canalul STOMP este ilustrat în Figura A6.4 din Anexa 6.

4.5 Izolarea datelor și modelul persistenței (database-per-service)

Arhitectura datelor se bazează pe fragmentarea persistenței în șapte scheme logice MySQL InnoDB complet izolate [17], abordare care limitează impactul defecțiunilor locale și evită situațiile în care un singur serviciu epuizează toate conexiunile disponibile la baza de date. Diagrama conceptuală globală a conexiunilor între scheme este detaliată în Figura A5.1 din Anexa 5.

Constrângerile rezultate din imposibilitatea JOIN-urilor inter-scheme sunt compensate prin două mecanisme structurale: tiparul Snapshot (detaliat în Secțiunea 4.5.2) și trasabilitatea prin super-clasa *BaseAuditableEntity*. Cheile externe fizice (*FOREIGN KEY*) sunt aplicate exclusiv în interiorul aceleiași scheme, beneficiind de suport tranzacțional și cascade automate. Pentru asocierile trans-schemă, platforma utilizează referințe aplicative logice bazate pe *keycloakId*, descris în Secțiunea 4.1.

Funcția de audit este standardizată prin super-clasa *BaseAuditableEntity*, care interceptează contextul de securitate curent și injectează automat marcajul temporal și UUID-ul operatorului (câmpurile *created_by* și *last_modified_by*) pentru orice operațiune de modificare a datelor medicale, asigurând trasabilitatea legală completă conform GDPR [1], independent de stratul de logică aplicativă.

4.5.1 Fundamentarea modelului relațional și nivelul de izolare

Datele gestionate de platforma KinetoCare necesită garanții de consistență imediată pentru operațiunile de rezervare și un regim strict de protecție pentru dosarele medicale, constrângeri care impun respectarea nativă a proprietăților ACID.

O decizie arhitecturală critică a fost configurarea explicită a nivelului de izolare la *READ_COMMITTED*, în detrimentul nivelului implicit *REPEATABLE READ*. Nivelul implicit utilizează mecanisme de tip *gap locks* care, în contextul rezervărilor clinice, ar bloca intervale de timp adiacente și ar genera frecvent erori de tip *deadlock*. Nivelul *READ_COMMITTED* elimină aceste blocaje de interval, permițând scrieri concurente imediate [17]. Consistența rezervărilor este garantată complementar prin strategii de blocare pesimistă la nivel de rând, exploatate prin suportul specificației JPA (*PESSIMISTIC_WRITE*), care se traduce la nivel de motor în instrucțiuni *SELECT ... FOR UPDATE* [17], asigurând izolarea tranzacției active până la *commit*.

4.5.2 Structura schemelor și normalizarea datelor

Structura detaliată a entităților și diagramele entitate-relație (ERD) pentru fiecare microserviciu sunt disponibile în Figura A5.2 din Anexa 5. Deciziile de modelare au urmărit respectarea formei normale 3 (3NF) și a formei normale Boyce-Codd (BCNF), conform cărora atributele depind exclusiv de cheia primară a entității.

Nucleul tranzacțional este reprezentat de schema *programari_db*, care co-localizează tabelele *programari*, *evaluari*, *evolutii* și *relatie_pacient_terapeut*. Această grupare permite ca inițierea unei evaluări și activarea relației terapeutice să se execute sub o singură graniță tranzacțională locală, eliminând necesitatea unor protocoale complexe de coordonare distribuită.

O abatere asumată de la principiile normalizării o reprezintă implementarea tiparului Snapshot [3] în tabela *programari*. Atributele comerciale (preț, durată, tip serviciu) sunt stocate redundant la momentul rezervării, prin încălcarea deliberată a dependenței funcționale dintre *serviciu_id* și *preț* — o cerință de corectitudine financiară, deoarece o modificare viitoare a tarifelor nu trebuie să altereze retroactiv istoricul programărilor finalizate.

Pentru a asigura integritatea contabilă, atributele financiare utilizează tipul *DECIMAL(10,2)*. Spre deosebire de tipurile în virgulă mobilă (*FLOAT*, *DOUBLE*), care pot acumula erori de rotunjire, formatul zecimal garantează precizia matematică absolută în operațiunile de agregare a veniturilor clinicii. Stările entităților sunt implementate prin tipul nativ *ENUM*, care oferă validare structurală eficientă și stocare compactă în spațiul paginilor de memorie [17].

4.5.3 Strategia de indexare

Optimizarea timpilor de răspuns s-a realizat prin definirea de indecși secundari adaptați tiparelor de interogare ale microserviciilor, exploatând structurile B+Tree ale motorului InnoDB [17].

Pentru interogarea critică ce validează disponibilitatea orară a fost creat indexul compus *idx_prog_overlap* pe coloanele (*terapeut_keycloak_id*, *data*, *ora_inceput*, *ora_sfarsit*). Respectând regula prefixului stâng, motorul izolează instantaneu sub-arborele terapeutului și execută o parcurgere eficientă doar pe intervalul vizat, eliminând necesitatea unei scanări complete de tabelă [17].

Interogările de raportare administrativă beneficiază de indexul de acoperire *idx_prog_stats*, definit pe (*locatie_id*, *data*, *pret*). Deoarece indexul conține toate coloanele necesare clauzelor WHERE și funcției SUM(), motorul extrage informația direct din *buffer pool*, ocolind complet operațiunile costisitoare de citire de pe disc [17].

În schema *chat_db*, extragerea istoricelor conversaționale este optimizată prin indexul *idx_mesaj_conv_list* pe coloanele (*conversatie_id*, *trimis_la*). Păstrând elementele deja ordonate cronologic în structura arborelui, se evită operația de sortare în memorie (*filesort*), garantând latențe reduse pentru volume mari de mesaje [17].

4.6 Reziliența aplicativă și toleranța la defecte

Platforma KinetoCare implementează reziliența pe două planuri arhitecturale complementare, ale căror fundamente tehnologice au fost justificate în Capitolul 3. La nivelul infrastructurii, mecanismele de auto-vindecare Kubernetes (*liveness* și *readiness probes*) [21], expuse prin *endpoint-urile* Spring Boot Actuator, garantează că traficul este dirijat exclusiv către instanțele complet inițializate și că procesele blocate sunt reciclate automat, fără intervenție manuală. La nivelul comunicării asincrone, reziliența față de mesajele structural defecte este asigurată prin topologia *dead letter queue* (DLQ). În acest sens, cozile principale direcționează mesajele eșuate (prin semnale NACK fără *requeue*) către o rețea de carantinare (*dead letter exchange* și *dead letter queue*). Consumatorul cozii de carantinare utilizează absorbția explicită a excepțiilor interne — o decizie deliberată pentru a preveni recursivitatea infinită a brokerului în cazul mesajelor structural toxice, protejând astfel disponibilitatea modulului de notificări față de defectele tranziente ale datelor. Versiunea actuală nu include mecanisme de reziliență

la nivel aplicativ, precum *circuit breaker* sau politici de *retry* cu întârziere progresivă, acestea constituind direcții de dezvoltare viitoare.

4.7 Topologia asincronă a evenimentelor de notificare

Infrastructura de mesagerie asincronă a modului *notificari-service* utilizează un *TopicExchange* RabbitMQ [9] cu o cheie de rutare de tip *wildcard* (*notificare.#*), permițând oricărui microserviciu producător să publice noi tipuri de evenimente fără a necesita reconfigurarea brokerului — o proprietate esențială pentru extensibilitatea sistemului. Topologia completă de carantinare a mesajelor eșuate este ilustrată în Figura A6.6 din Anexa 6, iar mecanismul de deduplicare prin idempotență este detaliat în Secțiunea 5.6.

Capitolul 5. Implementări Tehnice și Logica de Sistem

Acest capitol documentează mecanismele decizionale și arhitecturale centrale ale platformei KinetoCare. Sunt analizate soluțiile implementate pentru automatizarea fluxului clinic, generarea iterativă a ferestrelor de disponibilitate, prevenirea anomaliilor de interogare temporală, optimizarea agregării datelor inter-servicii, consistența asimetrică în operațiunile de dezactivare și garantarea idempotenței consumatorilor de mesaje.

5.1 Gestiunea traiectoriei clinice pe baza stării datelor

Pentru a elimina riscurile operaționale descrise anterior, asociate selecției manuale a procedurilor clinice, sistemul implementează o logică decizională de tip automat finit determinist (AFD). Acesta determină tipul de serviciu direct pe server, la fiecare inițiere a unei programări.

Sistemul operează prin trei stări clinice mutual exclusive, calculate dinamic. Starea de **evaluare inițială** este activă atunci când pacientul nu deține nicio evaluare înregistrată în baza de date; în acest caz, sistemul blochează orice altă procedură și alocă automat serviciul de evaluare. Starea de **tratament activ** devine curentă când pacientul deține o evaluare validă, iar interogarea bazei de date confirmă că numărul de ședințe finalizate este strict inferior cotei prescrise. În momentul în care bugetul de ședințe a fost epuizat, sistemul comută pe starea de **reevaluare**, respingând rezervarea unei noi ședințe de tratament standard și impunând procedura de reevaluare clinică.

O proprietate esențială a acestui mecanism este caracterul său determinist. Starea clinică a pacientului nu este stocată ca o variabilă independentă ce ar putea fi coruptă, ci este recalculată în timp real prin interogarea istoricului din persistența locală. Repornirile sau defecțiunile temporare ale microserviciului nu afectează traiectoria pacientului. La finalizarea unei noi reevaluări, sistemul derivă automat noua stare pe baza planului terapeutic actualizat, reluând ciclul, conform diagramei de tranziție din Figura A6.1, Anexa 6. Arhitectura decuplează intenționat automatizarea administrativă de autonomia medicală, permițând terapeutului să adauge o reevaluare timpurie direct din aplicație, înaintea epuizării planului curent. În acest scenariu, sistemul resetează dinamic contorul de ședințe conform noilor recomandări, restabilind sincronizarea progresului clinic.

5.2 Generarea ferestrelor de disponibilitate prin procesare iterativă

Calculul sloturilor valide pentru o programare în funcție de terapeut, locație și dată implică o serie de constrângeri cumulative: încadrarea în programul de lucru activ, absența concediilor și lipsa suprapunerilor cu alte rezervări confirmate din baza de date.

O decizie centrală de optimizare în proiectarea acestei metode a fost aplicarea principiului *fail-fast* în faza de colectare a datelor. Toate constrângerile și apelurile inter-servicii sunt executate defensiv înainte de procesarea efectivă a intervalelor. Dacă o interogare confirmă că terapeutul este în concediu în ziua solicitată, metoda returnează instantaneu o listă vidă, evitând interogarea agendei și consumul inutil de resurse computaționale. Fluxul este detaliat în Figura A6.2, Anexa 6.

Nucleul procesării utilizează o abordare iterativă de tip fereastră glisantă (*sliding window*). Un cursor orar avansează de la debutul programului de lucru cu un increment egal cu durata procedurii medicale, la care se adaugă un *buffer* operațional fix de 10 minute, necesar pentru igienizarea echipamentelor și redactarea notelor clinice. La fiecare iterație, intervalul generat este validat împotriva programărilor extrase din baza de date. Abordarea asigură o performanță ridicată, eliminând necesitatea utilizării unor algoritmi complecși de programare dinamică sau *backtracking*, justificată de faptul că deciziile de alocare a intervalelor sunt complet independente unele de altele.

5.3 Partiționarea temporală și anomaliile interogărilor la miezul nopții

Planificatorul platformei, implementat ca un proces asincron de fundal, transmite alerte automate cu 24 de ore, respectiv 2 ore înaintea programărilor, utilizând ferestre de căutare centrate pe momentul execuției. O problemă structurală apare atunci când execuția are loc în proximitatea miezului nopții. Spre exemplu, la ora 23:55, o marjă de căutare de ± 15 minute produce un interval logic care traversează granița zilei calendaristice: [23:40 — 00:10].

Separarea componentei orare de data calendaristică la nivelul persistenței face imposibilă utilizarea operatorului SQL *BETWEEN* pentru intervalele de la miezul nopții. Motorul bazei de date returnează un set vid deoarece evaluează condiția ca fiind falsă (valoarea 00:10 este inferioară matematic valorii 23:40). Această particularitate a designului de date ignoră silențios programările de la granița zilelor, problemă remediată tehnic prin aplicarea mecanismului de partiționare temporală direct din codul aplicației.

Pentru a asigura robustețea interogărilor, platforma implementează o tehnică de partiționare temporală binară. Atunci când logica aplicativă detectează că data de început a ferestrei diferă de data de sfârșit, intervalul este divizat în două interogări SQL complementare: prima acoperă intervalul de la începutul ferestrei de căutare până la sfârșitul zilei curente (23:59:59), iar a doua interoghează ziua următoare, de la miezul nopții (00:00:00) până la capătul superior al ferestrei. Rezultatele sunt concatenate direct în memorie, garantând, prin marje de suprapunere calibrate precis, că nicio programare nu este omisă din fluxul de notificare. Mecanismul de divizare a ferestrei temporale este ilustrat în Anexa 6, Figura A6.3.

5.4 Optimizarea agregării datelor și rezolvarea problemei N+1

Pentru a preveni latențele specifice problemei N+1 la nivel de rețea în procesul de asamblare a dosarului clinic integrat, extragerea datelor distribuite utilizează tehnica procesării în lot (*batch processing*). Detaliile entităților conexe stocate în microserviciile independente sunt preluate printr-un singur apel HTTP colectiv și structurate local într-un *hash map* pentru a permite accesul direct.

Performanța acestui flux este completată prin utilizarea abstracției Spring Cache, configurată cu un motor Caffeine și o fereastră de expirare de 10 minute. Acest strat memorează temporar datele cu volatilitate redusă (catalogul de locații, profilurile statice), eliminând apelurile inter-servicii redundante. De asemenea, fluxul aplică principiul degradării grațioase:

indisponibilitatea temporară a unui serviciu conex declanșează metode de *fallback*, precum afișarea identificatorilor bruți în locul numelor complete, garantând accesul neîntrerupt al personalului medical la datele clinice esențiale.

5.5 Consistența asimetrică la dezactivarea conturilor

În cadrul fluxului de dezactivare a unui cont, platforma aplică un tipar Saga de compensare. Modificarea stării contului se execută sincron atât în furnizorul de identitate Keycloak, cât și în baza de date locală. Deși abordează blocarea critică cu maximă prioritate, acest apel distribuit asimilează tehnic același risc rezidual de inconsistență descris în limitările sistemului (Capitolul 8). Pe de altă parte, anularea programărilor viitoare și actualizarea vizibilității în agendele clinicii tolerează modelul consistenței eventuale, operațiunile fiind propagate prin apeluri REST sincrone tratate defensiv — excepțiile de rețea sunt absorbite pentru a nu anula tranzacția principală de blocare a contului. Această abordare maximizează disponibilitatea operațiunilor de securitate fără a periclita integritatea datelor medicale. Fluxul de dezactivare cu consistență asimetrică este prezentat în Anexa 6, Figura A6.5.

5.6 Idempotența consumatorilor de mesaje

Un risc specific sistemelor distribuite asincrone este procesarea duplicată a evenimentelor, rezultată din suprapunerea ferestrelor de planificare sau din reîncercările de livrare la nivelul rețelei. Platforma tratează această problemă la nivelul bazei de date a consumatorului prin implementarea idempotenței operaționale. Fiecărui eveniment *i* se asociază la sursă un identificator unic. La recepție, sistemul încearcă persistența acestuia printr-o operațiune SQL atomică de tip inserare cu tratarea duplicatelor (*INSERT ... ON DUPLICATE KEY UPDATE*). Dacă motorul bazei de date raportează zero rânduri afectate din cauza constrângerii de unicitate a cheii, mesajul este clasificat ca duplicat și ignorat în siguranță. Acest mecanism garantează că pacienții nu primesc notificări multiple pentru același eveniment clinic, asigurând o stabilitate superioară a platformei în mediul de producție.

Capitolul 6. Interfața cu utilizatorul și fluxurile operaționale

Acest capitol descrie modul în care deciziile arhitecturale din nivelurile inferioare ale sistemului se reflectă în interfața grafică a platformei KinetoCare. Sunt analizate patru componente principale: experiența pacientului, acțiunile clinice ale terapeutului, panoul

administrativ centralizat și modulele de comunicare transversală (notificări și mesagerie în timp real), din perspectiva interacțiunii dintre componentele frontend (aplicația React [10]) și serviciile backend. Analiza evidențiază managementul stării, barierele de securitate și optimizarea cererilor HTTP.

6.1 Fluxul pacientului: de la profilare clinică la monitorizare

Profilarea inițială și barierele de rutare. Înregistrarea pacientului se desfășoară în două etape. Prima colectează datele de bază ale contului, iar a doua, completarea profilului medical, este obligatorie înainte de accesul la modulele operaționale. Funcționarea cu profiluri incomplete reprezintă un risc pentru validarea datelor și conformitatea legală, motiv pentru care această constrângere este aplicată activ printr-o barieră de rutare (*route guard*). Spre deosebire de starea de autentificare (gestionată global prin Context API), starea de completitudine a profilului este verificată la fiecare accesare a grupurilor de rute protejate printr-o interogare directă către serviciul de profil, funcționând ca o barieră în fluxul de navigare ce suspendă randarea componentei destinație până la obținerea răspunsului. Această barieră verifică prezența datelor medicale obligatorii (CNP, data nașterii) și, în caz de absență, redirecționează utilizatorul forțat către formularul de profilare, deblocând accesul la platformă exclusiv după confirmarea salvării cu succes a datelor.

Selectarea terapeutului și rezoluția datelor prin BFF. Interfața de căutare a terapeutilor necesită date agregate din mai multe surse: specializările, locațiile fizice, disponibilitatea și detaliile de identitate. Pentru a nu încărca clientul cu interogări multiple, cererea este preluată de API Gateway prin tiparul Backend-for-Frontend (BFF) [2]. Acesta lansează apeluri concurente către microservicii, aplică procesarea în lot pentru rezolvarea numelor și returnează un răspuns compact aplicației React.

Schimbarea terapeutului și arhivarea decuplată a relației clinice. Schimbarea terapeutului constituie un eveniment de domeniu critic cu o strategie de persistență deliberat decuplată. La nivelul interfeței, pacientul alege un nou specialist, iar în serviciul de pacienți câmpul de preferință terapeutică este actualizat imediat, programările viitoare cu fostul terapeut fiind anulate sincron. Totuși, relația terapeutică oficială înregistrată în baza de date a serviciului de programări nu este arhivată la momentul selectării, deoarece aceasta reflectă realitatea clinică dobândită, nu intenția declarată. Arhivarea relației precedente și activarea celei noi se produc decuplat, la primul eveniment clinic concret cu noul terapeut (finalizarea unei ședințe sau

salvarea unei evaluări clinice), moment în care sistemul închide automat legătura veche și o activează pe cea nouă. Această strategie protejează integritatea istoricului terapeutic în cazul în care pacientul revine asupra alegerii înainte de prima consultație. Din punct de vedere clinic, planul terapeutic activ este conservat: automatul finit determinist folosește cel mai recent formular de evaluare, indiferent de semnatar, permițând noului terapeut să preia recuperarea cu acces deplin la diagnostic și la bugetul de ședințe rămas.

Fluxul de rezervare și determinarea automată a serviciului. Interfața prezintă pacientului exclusiv intervalele orare valide, generate dinamic. Corpul cererii HTTP de rezervare conține doar coordonatele temporale și identificatorii resurselor; tipul de serviciu medical este omis intenționat, determinarea acestuia revenindu-i automatului finit din backend pentru a preveni selecțiile eronate. Aplicația actualizează starea locală fără reîncărcarea paginii și prezintă instantaneu un indicator vizual al evoluției planului clinic.

Jurnalul post-ședință. Introducerea metricilor subiectivi, adică intensitatea durerii, dificultatea exercițiilor și nivelul de oboseală, se realizează prin cursoare cu interval 1–10. Pacientul poate evalua retroactiv orice ședință neevaluată din istoric, selectând-o dintr-un meniu derulant, fără a fi constrâns de o ordine cronologică strictă. Harta de navigare a pacientului este prezentată în Figura A7.1 din Anexa 7.

6.2 Fluxul operațional al terapeutului: calendar și documentare

Configurarea profilului și managementul disponibilităților. Spre deosebire de fluxul restrictiv al pacientului, terapeutul nu este blocat fizic dacă profilul său este incomplet. La detectarea absenței unor date esențiale (specializarea sau orarul de lucru), panoul principal afișează o alertă persistentă și proeminentă. Aceasta informează terapeutul că profilul său este invizibil în listele de căutare ale pacienților până la remedierea lacunelor, dar permite în continuare accesul la calendar și la pacienții activi, asigurând continuitatea îngrijirii medicale. Suprapunerile de orar sunt prevenite activ: adăugarea zilelor de concediu invalidează automat acele date în algoritmi de programare prezentați pacienților.

Calendarul clinic cu preluare limitată la intervalul vizualizat. Descărcarea integrală a istoricului programărilor ar genera timpi de transfer ineficienți și un consum excesiv de memorie în browser. Componenta de calendar, implementată prin biblioteca FullCalendar [13], utilizează un tipar de preluare limitată la intervalul vizualizat (*viewport-based data fetching*):

la fiecare navigare între luni sau săptămâni, frontend-ul expediază exclusiv coordonatele vizibile, microserviciul returnând strict sub-setul de programări aferent. Logica de prezentare integrează reguli semantice clinice: programările anulate standard sunt ascunse, cele anulate prin neprezentare capătă un stil vizual estompat pe un fundal roșu, iar prima întâlnire cu un pacient nou este evidențiată vizual, semnalizând necesitatea evaluării inițiale.

Documentarea clinică și vizualizarea evoluției. Fișa digitală a pacientului agregă la cerere, prin apeluri HTTP sincrone către API Gateway, informațiile distribuite în ecosistem, fiind structurată în secțiuni clare: istoricul evaluărilor clinice, notele de evoluție și graficele evolutive. Notele de evoluție (observații clinice text-liber) sunt filtrate strict pentru autorul lor. Aceasta este o decizie arhitecturală deliberată, întrucât acestea reflectă perspectiva subiectivă a terapeutului și nu sunt partajate altor persoane, în timp ce diagnosticul obiectiv (evaluarea) rămâne mereu accesibil. Graficele suprapun pe o axă temporală comună evoluția durerii, oboselii și dificultății raportate de pacient în jurnale, transformând datele tabelare în tendințe vizuale (prin biblioteca Recharts [12]), permițând identificarea rapidă a deviațiilor de la traiectoria optimă. Fluxul operațional complet al terapeutului este prezentat în Figura A7.2 din Anexa 7.

6.3 Panoul de administrare și agregarea datelor statistice

Securitatea rolului administrativ și managementul catalogului. Accesul la panoul de bord administrativ este protejat la două niveluri: componenta de rutare protejată din frontend validează prezența rolului de administrator (extras din structura jetonului JWT în contextul global de autentificare), iar metodele din microserviciile destinație sunt protejate prin reguli de autorizare la nivel de endpoint (*requestMatchers*). Redundanța garantează că o eventuală eroare de configurare a interfeței nu compromite securitatea datelor de pe server, respectând principiul apărării stratificate (*defense in depth*). În modulele de gestiune a locațiilor și serviciilor, eliminarea resurselor utilizează exclusiv ștergerea logică (*soft-delete*). Comutarea unui indicator de inactivitate previne coruperea istoricului programărilor și încălcarea integrității referențiale în baza de date. Actualizările tarifare nu afectează contabilitatea trecută, datorită tiparului Snapshot (Secțiunea 4.5.2) aplicat la nivelul programărilor. Administratorul dispune și de un modul centralizat pentru suspendarea conturilor, acțiune care declanșează un proces în lanț de anulare a sesiunilor active și a programărilor viitoare.

Panoul de statistici și optimizarea performanței. Tabloul de bord administrativ necesită agregarea volumelor mari de date (programări per lună/locăție, venituri totale, terapeuți activi, rata de anulare). Aplicația React implementează două optimizări complementare pentru a preveni blocarea interfeței. O primă optimizare constă în concurența cererilor HTTP: la încărcarea ecranului de statistici, interfața lansează interogările simultan, reducând timpul total de încărcare la durata celui mai lent apel de rețea și evitând cumulara latențelor. Cea de-a doua optimizare vizează stocarea intermediară a rezultatelor prin intermediul hook-ului *useMemo*. Astfel, indicatorii cheie de performanță (venituri totale, rata de anulare, numărul de programări) sunt reevaluați exclusiv atunci când datele brute primite de la server sau filtrele active se modifică, eliminând recalculările redundante provocate de re-randările React. Datele agregate sunt predate bibliotecii vizuale Recharts [12], care randează grafice vectoriale scalabile. Spre deosebire de interfețele pacienților și terapeuților, care includ funcționalități reactive de timp real, panoul administrativ operează robust și exclusiv prin cereri HTTP REST sincrone. Topologia administrativă completă este prezentată în Figura A7.3 din Anexa 7.

6.4 Comunicarea transversală: notificări și mesagerie în timp real

Spre deosebire de managementul programărilor, care implică fluxuri operaționale profund asimetrice (pacientul rezervă, terapeutul execută și documentează), modulele de alerte și chat funcționează ca instrumente utilitare simetrice. Deoarece mecanismele de interacțiune asincronă și de timp real sunt tehnic și vizual identice pentru ambele roluri clinice, acestea sunt implementate și analizate ca funcționalități transversale (*cross-cutting concerns*) în arhitectura aplicației.

Sistemul de notificări și interogarea adaptivă. Componenta de notificări este integrată în bara de navigare globală și vizibilă în toate fluxurile platformei. Numărul de alerte necitite este actualizat periodic prin interogări HTTP (*polling*) către serviciul de notificări, o alternativă deliberată față de un canal WebSocket dedicat, al cărui cost de menținere nu ar fi justificat pentru frecvența redusă a acestor actualizări. Pentru a nu consuma resurse în perioadele de inactivitate, componenta utilizează API-ul nativ al browserului pentru vizibilitatea paginii: când utilizatorul comută pe un alt tab sau minimizează fereastra, interogările sunt oprite automat și reluate exclusiv la revenire.

Redirecționarea contextuală după rol. La accesarea meniului derulant, fiecare notificare afișată este acționabilă: un click o marchează automat ca citită și direcționează utilizatorul către

ecranul clinic relevant, determinat de rolul său. De exemplu, o alertă de mesaj nou trimite pacientul la modulul său de chat, iar terapeutul la interfața proprie de conversații, în timp ce o alertă de programare îi direcționează pe fiecare către calendarul specific. Această separare garantează că același eveniment produce o experiență contextuală corectă.

Mesageria clinică și protocolul STOMP. Canalul de mesagerie în timp real operează prin protocolul WebSocket/STOMP [14]. Fiind un spațiu comun, lista conversațiilor este construită printr-o fuziune de date: dacă o relație terapeutică este activă, dar nu are mesaje anterioare, sistemul randează dinamic o conversație virtuală. Persistența fizică în baza de date se declanșează abia la transmiterea primului mesaj (*lazy initialization*). Validitatea relației terapeutice este verificată sincron la fiecare mesaj expedit; dacă relația a fost arhivată (de exemplu, pacientul a ales alt specialist), transmisia este respinsă la nivel de backend. În acel moment, interfața degradează conversația într-un format strict de citire (*read-only*) pentru ambii utilizatori, securizând astfel granițele clinice. Diagrama de secvență pentru validarea mesajelor STOMP și propagarea asincronă a notificărilor este ilustrată în Figura A7.4 din Anexa 7.

Capitolul 7. Validare, Testare și Performanță

Într-o arhitectură distribuită, corectitudinea izolată a unui serviciu este o condiție necesară, dar insuficientă. Defectele cele mai grave apar la granițele dintre comportamente: consumatori concurenți, mesagerie asincronă sau servicii de identitate externe cuplate cu baze de date locale. Strategia de validare implementată în platforma KinetoCare este proiectată explicit în jurul acestor interfețe critice, acoperind patru niveluri fundamentale: corectitudinea algoritmică, rezistența la concurență tranzacțională, comportamentul fluxurilor distribuite și reziliența infrastructurii containerizate.

7.1 Testarea unitară a componentelor algoritmice critice

Automatul finit clinic și algoritmul de disponibilitate cu fereastră glisantă sunt testate în izolare completă față de infrastructură. Toate dependențele externe ale serviciului de programări sunt înlocuite cu dubluri de test prin intermediul framework-ului Mockito, fără pornirea unui context Spring.

Automatul finit determinist este acoperit prin trei cazuri ce traversează integral stările posibile ale fluxului clinic: absența oricărei evaluări anterioare determină solicitarea automată a uneia

inițiale; existența unei evaluări active cu ședințe sub cota recomandată (5 din 10) conduce la preluarea serviciului de tratament standard; epuizarea completă a cotei (10 din 10) declanșează tranziția automată spre serviciul de reevaluare obligatorie.

Algoritmul cu fereastră glisantă este evaluat pe un scenariu definit de intervalul operațional 08:00–12:00, un serviciu de 60 de minute și o rezervare preexistentă pe 09:00–10:00. Execuția generează exact două sloturi disponibile — 08:00 și 10:20 — confirmând logica de includere a intervalelor, mecanismul de avansare cu buffer și oprirea corectă la limita superioară a zilei. Rezultatul acestei execuții este ilustrat în Anexa 8, Figura A8.1.

7.2 Testarea de integrare a concurenței tranzacționale

Spre deosebire de componentele algoritmice, mecanismele de concurență și izolare tranzacțională nu pot fi validate prin dubluri de test, deoarece acestea elimină comportamentul nativ al motorului bazei de date. Clasa *DoubleBookingIntegrationTest* utilizează un context Spring complet, conectat direct la o instanță reală de MySQL.

Testul demonstrează că mecanismul de blocare pesimistă (Secțiunea 4.5.1) previne înregistrarea concurentă a două programări suprapuse. Sunt lansate simultan 50 de fire de execuție, fiecare simulând o cerere de rezervare pentru același interval orar și același terapeut. Sincronizarea pornirii este garantată printr-o barieră de tip *CountDownLatch*: toate firele sunt inițializate și suspendate la un punct comun, eliberate simultan printr-un singur semnal pentru a forța o condiție de cursă maximală. Aserțiunile finale validează patru condiții: exact un fir reușește, exact 49 eșuează cu conflict, zero erori neașteptate și o singură înregistrare validă în baza de date. Distribuția rezultatelor obținute este prezentată în Anexa 8, Figura A8.2.

O constrângere specifică acestui test o reprezintă incompatibilitatea cu strategia standard de rollback automat din Spring. Contextul tranzacțional este mapat la nivel de *ThreadLocal*, astfel că firele secundare nu moștenesc tranzacția firului principal. Curățarea stării între rulări se realizează manual, respectând ierarhia impusă de integritatea referențială InnoDB: înregistrările din tabela de evaluări sunt șterse înaintea celor din tabela de programări.

7.3 Validarea funcțională a infrastructurii distribuite

Topologia asincronă din RabbitMQ (Secțiunea 4.6) este validată prin injectarea deliberată a unui mesaj JSON malformat. Comportamentul observat confirmă scenariul de reziliență

proiectat: excepția de deserializare determină emiterea unui NACK fără reintroducere în coadă (*requeue*), urmată de rutarea automată a sarcinii utile (*payload-ului*) afectate spre coada de carantinare (*dead letter queue*), coada principală continuând procesarea mesajelor valide fără degradări. Evidența acestui comportament este inclusă în Anexa 8, Figura A8.3.

Securizarea canalului în timp real prin *StompSecurityInterceptor* (Secțiunea 4.4) este evaluată prin două scenarii complementare. Cu JWT valid, clientul traversează interceptorul cu succes, contextul de securitate fiind propagat către componentele din aval. Cu jeton expirat sau alterat structural, interceptorul curăță contextul de securitate al firului, conexiunea WebSocket este respinsă la nivel de protocol, iar un mesaj de eroare structurat este rutat pe coada utilizatorului; interfața React blochează persistența datelor și afișează un banner informativ. Scenariul de respingere a conexiunii este prezentat în Anexa 8, Figura A8.4.

Scenariul de eșec parțial din fluxul de înregistrare (Secțiunea 3.2) este simulat prin oprirea forțată a microserviciului de profil. Eșecul scrierii locale activează blocul compensatoriu, care revocă operațiunea prin ștergerea contului generat în Keycloak, iar excepția propagată declanșează *rollback-ul* persistenței locale. Rezultatul final confirmă că utilizatorul parțial înregistrat nu există în niciunul dintre mediile de stocare. Confirmarea stării finale este prezentată în Anexa 8, Figura A8.5.

7.4 Validarea rezilienței în interfața utilizatorului

Componenta de notificări interoghează backend-ul la un interval fix de 30 de secunde. Integrarea cu Page Visibility API adaptează acest mecanism la comportamentul utilizatorului: dezactivarea tab-ului suspendă intervalul de polling, iar reactivarea paginii declanșează o interogare imediată de sincronizare, după care ritmul regulat este reluat. Funcția de curățare a componentei React eliberează atât ascultătorul de eveniment, cât și intervalul activ la demontare, prevenind scurgerile de memorie. Comportamentul componentei în raport cu starea de vizibilitate a paginii este ilustrat în Anexa 8, Figura A8.6.

Componenta *ErrorBoundary* interceptează erorile de runtime JavaScript la nivel global, înlocuind ecranul complet alb cu o interfață de repliere și un mecanism de reîncărcare a paginii, prevenind degradarea silențioasă a experienței utilizatorului. Exemplul de repliere a interfeței este prezentat în Anexa 8, Figura A8.7.

7.5 Validarea rezilienței în infrastructura bazată pe containere

Funcționarea mecanismelor *readiness* și *liveness probe* (Secțiunea 4.6) este validată prin monitorizarea ciclului de viață al containerelor în faza de pornire. *Readiness* au izolat pod-urile aflate în inițializare, prevenind direcționarea traficului prin API Gateway până la stabilirea conexiunilor la bazele de date. Testarea sondelor *liveness* prin blocarea artificială a proceselor din mașina virtuală Java a confirmat capacitatea de autoregenerare, clusterul Kubernetes executând repornirea automată a containerelor compromise. Ciclul de detectare și repornire este evidențiat în Anexa 8, Figura A8.8.

Comportamentul platformei în scenariul degradării grațioase este evaluat printr-o configurație Kubernetes care rulează deliberat doar subsetul principal al microserviciilor — autentificare, profil, gateway și frontend — excluzând serviciile secundare din considerente de resurse locale. La oprirea intenționată a microserviciului de programări, decuplarea arhitecturală operează pe două niveluri: backend-ul absoarbe excepțiile de rețea prin mecanisme de *timeout*, iar componentele React interceptează erorile HTTP prin *ErrorBoundary*. Modulele de autentificare și profil rămân complet funcționale, impactul fiind izolat sub forma unui mesaj de avertizare localizat în zona calendarului, confirmând reziliența globală a platformei sub constrângeri de infrastructură. Manifestarea degradării grațioase la nivelul interfeței este prezentată în Anexa 8, Figura A8.9.

Capitolul 8. Limitări arhitecturale și direcții viitoare

Pe parcursul dezvoltării platformei KinetoCare au fost asumate deliberat o serie de compromisuri tehnice, justificate de contextul unui produs minim viabil (MVP). Secțiunile următoare documentează fiecare datorie tehnică și propun traiectoria de evoluție necesară pentru transformarea platformei într-o soluție de nivel enterprise, comercializabilă ca serviciu (SaaS). O limitare transversală asumată este absența unui ciclu extensiv de testare cu utilizatori reali din domeniu, ceea ce limitează certitudinea că fluxurile implementate sunt pe deplin adaptate nevoilor clinice efective.

8.1 Limitări arhitecturale și tehnice asumate

Consistența distribuită și riscul rezidual al scrierilor duale. Procesul de înregistrare, implementat ca tipar Saga cu compensare sincronă (Secțiunea 3.2), prezintă o fereastră de

inconsistență reziduală: dacă apelul compensatoriu spre Keycloak eșuează, de exemplu, din cauza unui *timeout* de rețea, tranzacția MySQL locală este anulată, dar contul din furnizorul de identitate (*identity provider*) rămâne activ, generând conturi inconsistente între sisteme. Frecvența de apariție este neglijabilă în contextul curent, iar jurnalizarea explicită a evenimentului oferă un punct de audit clar pentru curățarea manuală a conturilor reziduale.

Asamblarea secvențială a dosarului clinic. O optimizare importantă a fost realizată prin introducerea unui *endpoint batch* pentru încărcarea identităților terapeuților (Secțiunea 5.4). Datoria tehnică reziduală persistă în apelurile individuale pentru traducerea identificatorilor de servicii recomandate și pentru construcția listei de pacienți ai unui terapeut. Impactul este redus în contextul curent, însă această asamblare devine un blocaj semnificativ în scenariile cu date longitudinale masive.

Stocarea resurselor binare în baza de date relațională. Fotografiile de profil ale terapeuților sunt stocate ca șiruri Base64 direct în schema relațională, constituind o practică nerecomandată (*anti-pattern*) arhitectural. Datele binare fragmentează paginile InnoDB, concurează pentru memoria *buffer pool-ului* și măresc dimensiunea răspunsurilor API.

Absența modulelor de integrare fiscală. Integrarea cu sistemele naționale (e-Factura ANAF sau decontările CNAS) presupune redesenarea granițelor de domeniu, nu simpla adăugare de tabele. Un modul fiscal funcțional ar impune un microserviciu izolat, decuplat asincron de componenta de programări — decuplare obligatorie pentru a garanta că disponibilitatea redusă a API-urilor guvernamentale nu blochează fluxul clinic principal.

Absența unui instrument de versionare a schemelor relaționale. Inițializarea schemelor se realizează prin scripturi statice la crearea volumelor Docker, integritatea fiind verificată la pornire prin Hibernate. Această abordare este suficientă în dezvoltare, însă într-un mediu de producție, unde datele clinice sunt ireversibile, absența unui instrument de migrare incrementală (Flyway sau Liquibase) devine un risc operațional major.

Operațiuni sincrone de I/O în interiorul granițelor tranzacționale. Crearea unei programări execută apeluri de rețea sincrone prin OpenFeign în interiorul unei tranzacții active, reținând conexiunea din pool-ul HikariCP pe toată durata apelurilor externe. Sub sarcină ridicată, această secvență poate epuiza complet pool-ul de conexiuni, determinând eșecuri în cascadă.

8.2 Direcții viitoare de dezvoltare și scalare

Evoluția către o arhitectură multi-tenant. Arhitectura curentă deservește o singură clinică (*single-tenant*). Extinderea națională presupune adoptarea modelului prin strategia *schema-per-tenant*: la înrolarea unei clinici noi, sistemul generează dinamic o schemă MySQL dedicată, izolând datele la nivel fizic și respectând exigențele Art. 9 GDPR. La nivelul furnizorului de identitate, fiecare tenant va beneficia de un Realm Keycloak dedicat.

Refactorizarea integrării prin tiparul transactional outbox. Eliminarea riscului conturilor fantomă se realizează prin tiparul *transactional outbox*: serviciul de identitate va salva datele utilizatorului împreună cu un eveniment de creare într-o tabelă locală *outbox*, în cadrul aceleiași tranzacții MySQL. Un proces de fundal (sau un instrument *change data capture* precum Debezium) va propaga garantat evenimentul către Keycloak, asigurând consistența absolută.

Asistență AI pentru documentarea clinică. Direcția propusă integrează un model lingvistic de mari dimensiuni (LLM) capabil să structureze automat observațiile terapeutului. Din cauza restricțiilor GDPR privind transferul datelor de sănătate către API-uri comerciale terțe, soluția necesită un model cu cod sursă deschis (*open-source*) găzduit intern. Integrarea va fi asincronă, rezultatul fiind returnat exclusiv pentru validarea de către un specialist uman, asigurând trasabilitatea actului decizional clinic.

Externalizarea stocării media. Decuplarea imaginilor de baza de date relațională se va realiza prin integrarea unui serviciu de stocare compatibil cu protocolul S3 (o instanță MinIO găzduită intern). Coloana Base64 va fi înlocuită cu o referință URL externă, eliberând *buffer pool*-ul InnoDB, reducând dimensiunea răspunsurilor API și deschizând infrastructura pentru atașarea documentelor clinice complexe.

Capitolul 9. Concluzii

Implementarea platformei KinetoCare a confirmat că digitalizarea fluxurilor unui cabinet de kinetoterapie presupune mai mult decât transpunerea în software a unui calendar și a unui dosar medical. Principala dificultate nu a constat în volumul de funcționalități, ci în gestionarea corectitudinii unui sistem format din șapte microservicii de domeniu, fiecare cu propria persistență, și un API Gateway *stateless*, expuse riscului de inconsistență specific sistemelor distribuite, absent în arhitecturile monolitice.

Cele mai solicitante implementări au fost componentele algoritmice din Capitolul 5. Automatul finit care determină tipul de serviciu clinic a necesitat atenție deosebită la tranzițiile de margine, în special scenariul reevaluării timpurii care resetează contorul de ședințe fără să piardă istoricul. Algoritmul de disponibilitate orară a ridicat problema interogărilor SQL la granița zilei calendaristice, care returnează silențios un set vid la miezul nopții, fără semnalarea unei excepții la nivel aplicațional. Extinderea modelului Zero-Trust pe canalele WebSocket a impus un interceptor STOMP dedicat cu curățare explicită a contextului de securitate — o problemă care nu apare în arhitecturile HTTP clasice.

Din perspectiva contribuțiilor, platforma acoperă funcționalitățile absente din soluțiile analizate: determinarea automată a tipului de serviciu clinic prin automatul finit, mesageria directă pacient-terapeut și colectarea feedback-ului periodic al pacientului, mecanisme care transformă rezervarea dintr-un act administrativ într-un pas integrat în protocolul terapeutic. Totuși, limitările principale rămân absența modulelor de facturare și decontare, asumată ca decizie de prioritizare a unui MVP, alături de necesitatea validării empirice a fluxurilor cu utilizatori reali.

Deși proiectată pentru kinetoterapie, arhitectura platformei nu este dependentă de specificul acestei discipline. Orice ramură medicală cu protocoale iterative — stomatologie, psihoterapie, nutriție clinică — ar putea beneficia de același model: un motor decizional bazat pe istoricul pacientului, un canal de comunicare securizat terapeut-pacient și o buclă de feedback pe durata tratamentului. Reconfigurarea s-ar limita la ajustarea tranzițiilor din automatul finit și a catalogului de servicii clinice, celelalte mecanisme rămânând neschimbate.

Bibliografie

- [1] Parlamentul European și Consiliul Uniunii Europene, „Regulamentul (UE) 2016/679 privind protecția persoanelor fizice în ceea ce privește prelucrarea datelor cu caracter personal (GDPR)”, Jurnalul Oficial al Uniunii Europene, 2016.
- [2] S. Newman, Building Microservices: Designing Fine-Grained Systems, Ediția a 2-a. O'Reilly Media, 2021.
- [3] C. Richardson, Microservices Patterns: With examples in Java. Manning Publications, 2018.
- [4] S. Rose, O. Borchert, S. Mitchell, și S. Connelly, „Zero Trust Architecture”, National Institute of Standards and Technology (NIST), Special Publication 800-207, 2020.
- [5] M. Jones, J. Bradley, și N. Sakimura, „JSON Web Token (JWT)”, Internet Engineering Task Force (IETF), RFC 7519, Mai 2015. Disponibil: <https://datatracker.ietf.org/doc/html/rfc7519> [accesat: 29.05.2026]
- [6] Oracle Corporation, „The Java Language Specification, Java SE 21 Edition”. Disponibil: <https://docs.oracle.com/javase/specs/> [accesat: 29.05.2026]
- [7] C. Walls, Spring in Action, Ediția a 6-a. Manning Publications, 2022.
- [8] E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley Professional, 2003.
- [9] RabbitMQ, „RabbitMQ Documentation: AMQP 0-9-1 Model Explained”. Disponibil: <https://www.rabbitmq.com/tutorials/amqp-concepts.html> [accesat: 29.05.2026]
- [10] Meta Platforms, Inc., „React – A JavaScript library for building user interfaces”. Disponibil: <https://react.dev> [accesat: 29.05.2026]
- [11] Remix Software, „React Router Documentation”. Disponibil: <https://reactrouter.com/> [accesat: 29.05.2026]
- [12] Recharts, „Recharts: A composable charting library built on React components”. Disponibil: <https://recharts.org/> [accesat: 29.05.2026]
- [13] Adam Shaw, „FullCalendar React Component”. Disponibil: <https://fullcalendar.io/docs/react> [accesat: 29.05.2026]

- [14] I. Fette și A. Melnikov, „The WebSocket Protocol”, IETF, RFC 6455, Decembrie 2011. Disponibil: <https://datatracker.ietf.org/doc/html/rfc6455> [accesat: 29.05.2026]
- [15] STOMP, „STOMP Protocol Specification, Version 1.2”. Disponibil: <https://stomp.github.io/stomp-specification-1.2.html> [accesat: 29.05.2026]
- [16] SockJS, „SockJS Client Documentation”. Disponibil: <https://github.com/sockjs/sockjs-client> [accesat: 29.05.2026]
- [17] Oracle Corporation, „MySQL 8.0 Reference Manual”. Disponibil: <https://dev.mysql.com/doc/refman/8.0/en/> [accesat: 29.05.2026]
- [18] D. Hardt, „The OAuth 2.0 Authorization Framework”, IETF, RFC 6749, Octombrie 2012. Disponibil: <https://datatracker.ietf.org/doc/html/rfc6749> [accesat: 29.05.2026]
- [19] Red Hat, „Keycloak Documentation”. Disponibil: <https://www.keycloak.org/documentation> [accesat: 29.05.2026]
- [20] Docker Inc., „Docker Documentation”. Disponibil: <https://docs.docker.com/> [accesat: 29.05.2026]
- [21] K. Hightower, B. Burns, și J. Beda, Kubernetes: Up and Running, Ediția a 3-a. O'Reilly Media, 2022.
- [22] C. Bușu et al., „Digital Transformation of Medical Services in Romania: Does the Healthcare System Meet the Current Needs of Patients?”, Healthcare, vol. 13, nr. 20, 2025. Disponibil: <https://doi.org/10.3390/healthcare13202549>
- [23] WebPT, „The Leading Physical Therapy Software & Rehab Therapy EMR”. Disponibil: <https://www.webpt.com> [accesat: 29.05.2026]
- [24] Pabau, „Practice Management Software for Clinics & Medical Spas”. Disponibil: <https://pabau.com> [accesat: 29.05.2026]
- [25] FizioHub, „Platformă digitală pentru fizioterapeuți și clinici de recuperare”. Disponibil: <https://fiziohub.ro> [accesat: 29.05.2026]
- [26] ReKi, „Platformă SaaS pentru clinici de recuperare medicală, fizioterapie și kinetoterapie”. Disponibil: <https://rekiapp.ro> [accesat: 29.05.2026]
- [27] S. Gilbert și N. Lynch, „Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,” ACM SIGACT News, vol. 33, nr. 2, 2002.

Anexe

Anexa 1. Tabel comparativ al funcționalităților soluțiilor software existente

Tabelul de mai jos sintetizează funcționalitățile identificate în urma analizei documentației oficiale și a materialelor publice ale soluțiilor software evaluate. Evaluarea are rol comparativ și justifică cerințele funcționale ale platformei KinetoCare.

Legendă:

- Integrat = funcționalitate disponibilă în mod explicit în produs
- Parțial = funcționalitate disponibilă limitat sau prin module/extensii
- Absent = funcționalitate neidentificată în documentația publică

Criteriu de evaluare	WebPT	Pabau	FizioHub	ReKi	KinetoCare
Gestiunea programărilor	Integrat	Integrat	Integrat	Integrat	Integrat
Calendar interactiv	Integrat	Integrat	Integrat	Integrat	Integrat
Notificări și remindere automate	Integrat ¹	Integrat	Parțial	Parțial	Integrat
Documentare clinică	Integrat	Integrat	Integrat	Integrat	Integrat
Portal pacient	Integrat	Integrat	Integrat	Integrat	Integrat
Mesagerie pacient-terapeut	Absent	Absent	Absent	Absent	Integrat
Feedback post-ședință	Absent	Absent	Absent	Absent	Integrat
Plan terapeutic cu număr de ședințe	Absent	Absent	Parțial	Parțial	Integrat
Automatizare selecție serviciu clinic	Absent	Absent	Absent	Absent	Integrat
Alertă epuizare plan terapeutic	Absent	Absent	Absent	Absent	Integrat
Facturare / decontare	Integrat	Integrat	Parțial	Integrat	Absent ²
Asistență AI pentru documentare	Integrat ¹	Integrat	Absent	Absent	Absent ²
Adaptare la piața românească	Absent	Parțial	Integrat	Integrat	Integrat
Conformitate GDPR ³	Parțial	Parțial	Integrat	Integrat	Integrat

¹ Notă: funcționalitate disponibilă prin integrări de tip partener sau module adiționale, conform documentației oficiale. În cazul WebPT, asistența AI pentru documentare este oferită prin integrările cu PredictionHealth (2024) și Comprehend Health (2025); notificările automate sunt disponibile ca modul adițional, neinclus în planul de bază.

² Notă: funcționalitate absentă în implementarea actuală KinetoCare.

³Notă: Evaluarea conformității GDPR se bazează exclusiv pe funcționalitățile documentate public la data analizei și nu constituie o evaluare juridică a produselor respective. Ratingul Parțial pentru soluțiile internaționale reflectă absența unor mecanisme explicit descrise în documentația oficială pentru contextul legislativ european (control granular al accesului, jurnal de audit imutabil).

Anexa 2. Registrul microserviciilor platformei KinetoCare

Platforma este compusă din opt unități independente de *deployment*, detaliate conform responsabilităților lor:

Microserviciu	Port	Responsabilitate de domeniu	Schemă dedicată
<i>api-gateway</i>	8081	Rutare, proxy token și agregare BFF reactivă	<i>Stateless</i>
<i>user-service</i>	8082	Identitate, RBAC și management Keycloak	<i>user_db</i>
<i>pacienti-service</i>	8083	Profil medical extins și jurnal de feedback subiectiv	<i>pacienti_db</i>
<i>terapeuti-service</i>	8084	Resurse clinice, disponibilitate și concedii	<i>terapeuti_db</i>
<i>programari-service</i>	8085	Nucleu tranzacțional, evaluări clinice și relații	<i>programari_db</i>
<i>servicii-service</i>	8086	Catalog de servicii medicale și tarifyare	<i>servicii_db</i>
<i>chat-service</i>	8087	Comunicație bidirecțională în timp real (STOMP)	<i>chat_db</i>
<i>notificari-service</i>	8088	Ingestie asincronă de evenimente și alerte	<i>notificari_db</i>

Notă: Pentru consistență în mediul containerizat și pentru separarea clară între serviciile de aplicație și persistența datelor, fiecare microserviciu utilizează o schemă de bază de date dedicată, denumită conform convenției **_db* (ex. *user_db*).

Anexa 3. Matricea de comunicare inter-servicii (Sincron și Asincron)

Această anexă centralizează fluxurile de comunicare la nivelul platformei, atât pentru operațiunile tranzacționale blocante (OpenFeign), cât și pentru arhitectura orientată pe evenimente (RabbitMQ).

Tabel A3.1. Comunicații sincrone (Contracte OpenFeign)

Serviciu apelant	Serviciu destinație	Justificarea apelului sincron
<i>user-service</i>	<i>pacienti-service</i>	Inițializare profil la înregistrare; propagare stare la dezactivarea contului.
<i>user-service</i>	<i>terapeuti-service</i>	Inițializare profil la înregistrare; propagare stare la dezactivarea contului.
<i>user-service</i>	<i>programari-service</i>	Anularea programărilor viitoare la dezactivarea unui cont.
<i>programari-service</i>	<i>servicii-service</i>	Denormalizarea prețului și duratei la momentul rezervării (Tiparul Snapshot).
<i>programari-service</i>	<i>terapeuti-service</i>	Validarea orarului și a conflictelor cu perioadele de concediu.
<i>programari-service</i>	<i>user-service</i>	Rezolvarea numelor pacienților și terapeuților pentru fișa clinică.
<i>programari-service</i>	<i>pacienti-service</i>	Interogarea istoricului de jurnale și a datelor medicale ale pacientului în scopul agregării fișei clinice.
<i>pacienti-service</i>	<i>programari-service</i>	Declanșarea cascadei de anulare a programărilor viitoare și arhivarea (dezactivarea) relației active cu fostul terapeut.
<i>chat-service</i>	<i>programari-service</i>	Validarea relației terapeutice active anterior expedierii unui mesaj de chat.

Tabel A3.2. Comunicații asincrone (Evenimente AMQP)

Serviciu producător	Cheie de rutare semantică	Fapt împlinit (eveniment)
<i>programari-service</i>	<i>notificare.programare.noua</i>	Confirmarea unei rezervări de slot clinic.
<i>programari-service</i>	<i>notificare.evaluare.initiala</i>	Alocarea unei programări de tip Evaluare Inițială.
<i>programari-service</i>	<i>notificare.reevaluare.necesara</i>	Alocarea unei programări de tip Reevaluare.
<i>programari-service</i>	<i>notificare.programare.anulata.pacient</i>	Invalidarea unui slot din inițiativa pacientului.
<i>programari-service</i>	<i>notificare.programare.anulata.terapeut</i>	Invalidarea unui slot din inițiativa terapeutului.
<i>programari-service</i>	<i>notificare.reminder.jurnal</i>	Declanșarea ferestrei de colectare a feedback-ului după finalizarea ședinței.
<i>programari-service</i>	<i>notificare.reminder.24h</i>	Alertă temporală transmisă cu 24 de ore înaintea începerii programării.
<i>programari-service</i>	<i>notificare.reminder.2h</i>	Alertă temporală transmisă cu 2 ore înaintea începerii programării.
<i>programari-service</i>	<i>notificare.reevaluare.recomandata</i>	Notificarea pacientului că a atins pragul maxim de ședințe și necesită reevaluare.
<i>pacienti-service / programari-service</i>	<i>notificare.jurnal.completat</i>	Notificarea terapeutului că pacientul a completat jurnalul asociat ședinței.
<i>chat-service</i>	<i>notificare.mesaj.nou</i>	Notificarea utilizatorului (pacient sau terapeut) că a primit un mesaj nou într-o conversație activă.

Anexa 4. Arhitectura serviciilor și a fluxurilor de rețea

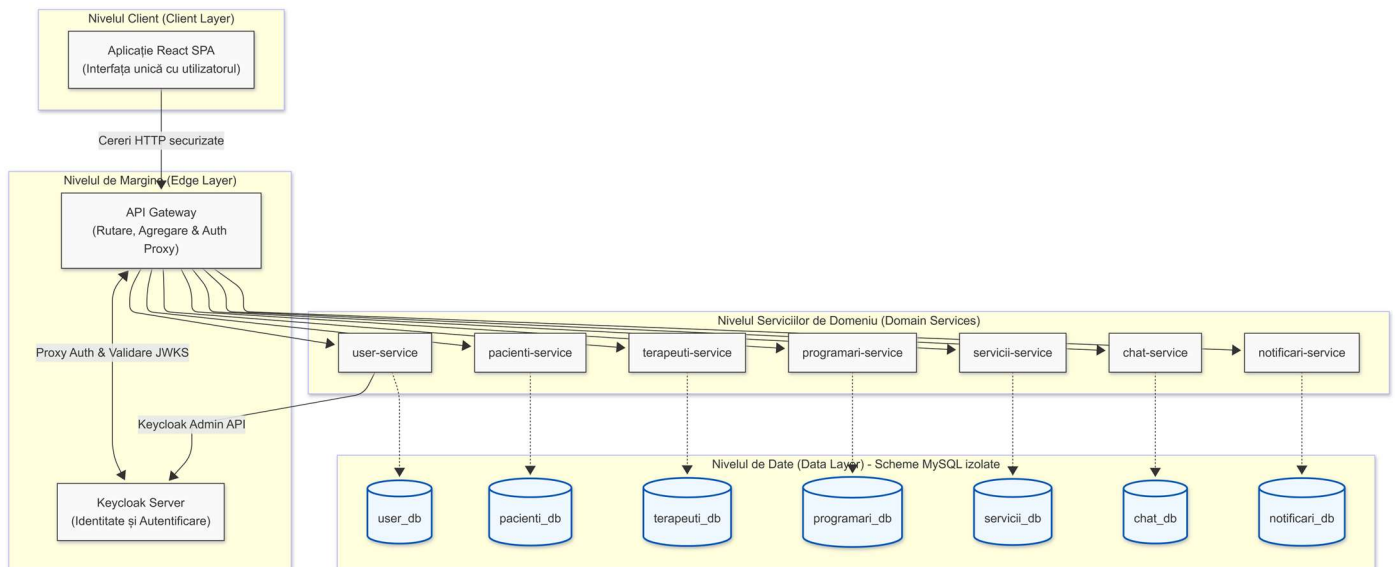


Figura A4.1. Topologia generală a arhitecturii microserviciilor pe niveluri de execuție

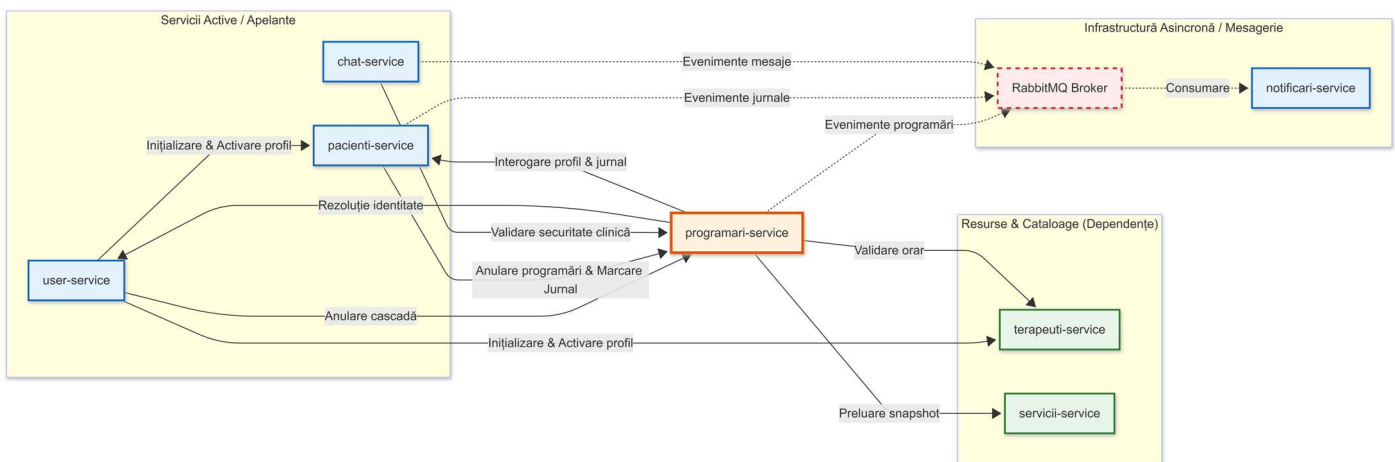


Figura A4.2. Graful orientat al fluxurilor de comunicare inter-servicii

Anexa 5. Arhitectura și modelarea bazelor de date

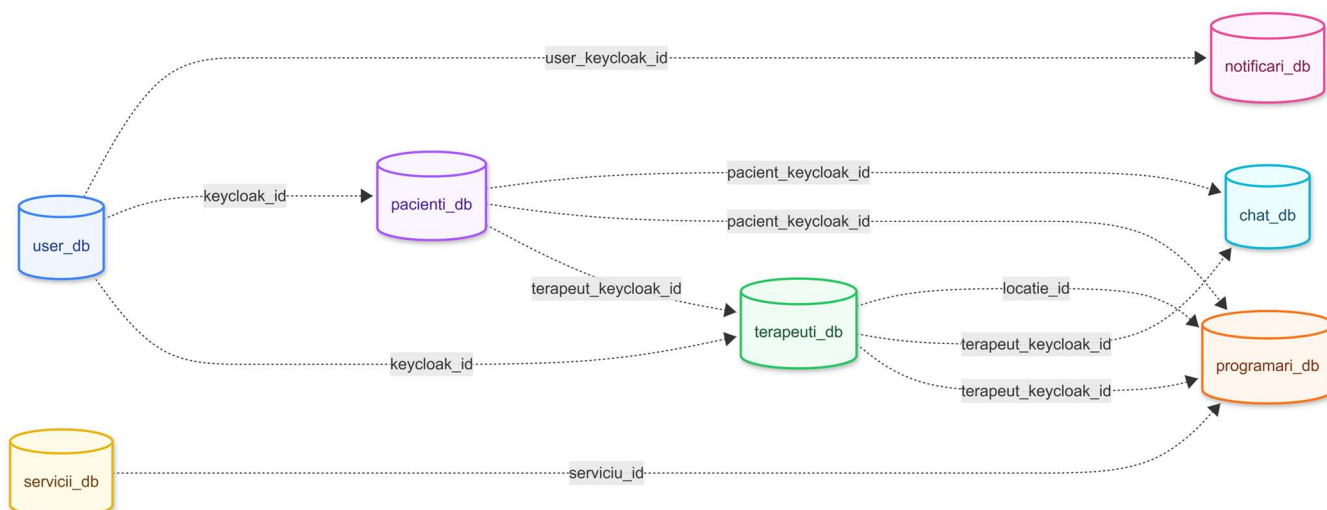


Figura A5.1. Diagrama conceptuală globală a conexiunilor logice între schemele bazei de date

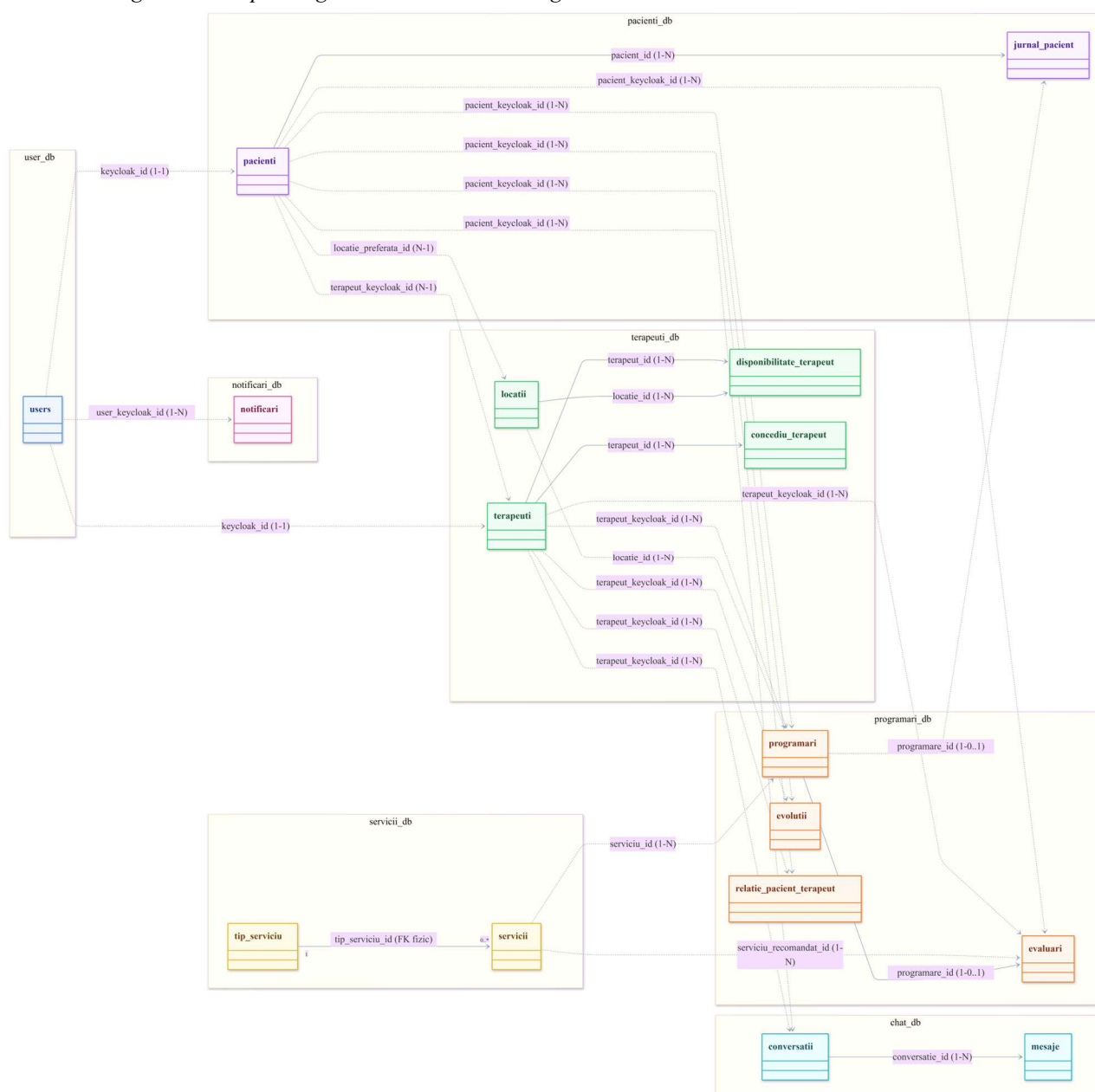


Figura A5.2. Structura detaliată Entitate-Relație (ERD) a bazelor de date

Anexa 6. Diagrame tehnice și logica de sistem

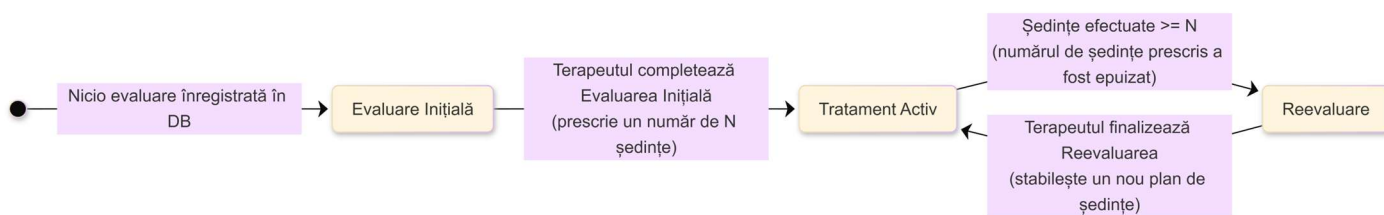


Figura A6.1. Diagrama de tranziție a stărilor clinice (Automat Finit Determinist)

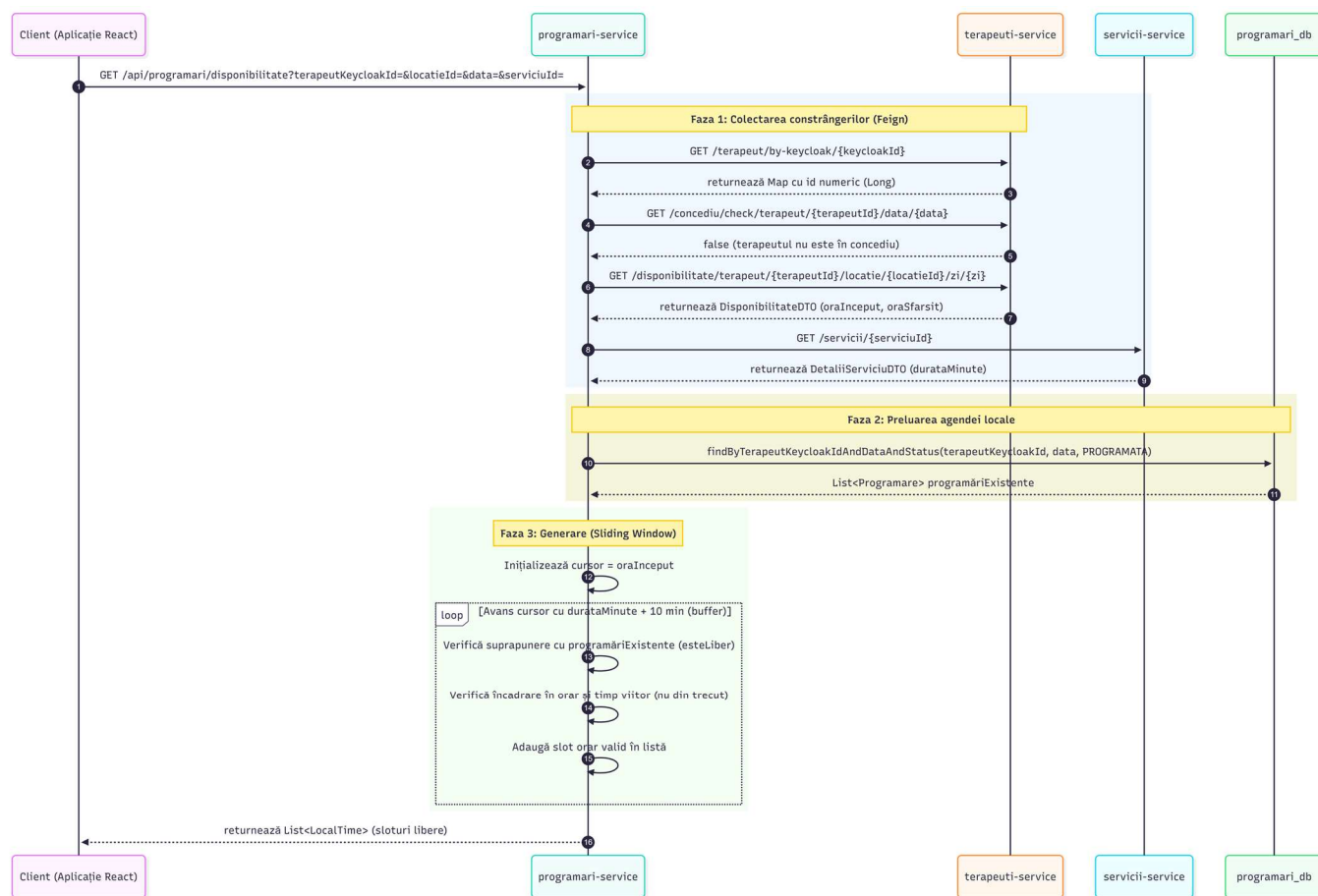


Figura A6.2. Fluxul secvențial de generare a ferestrelor de disponibilitate

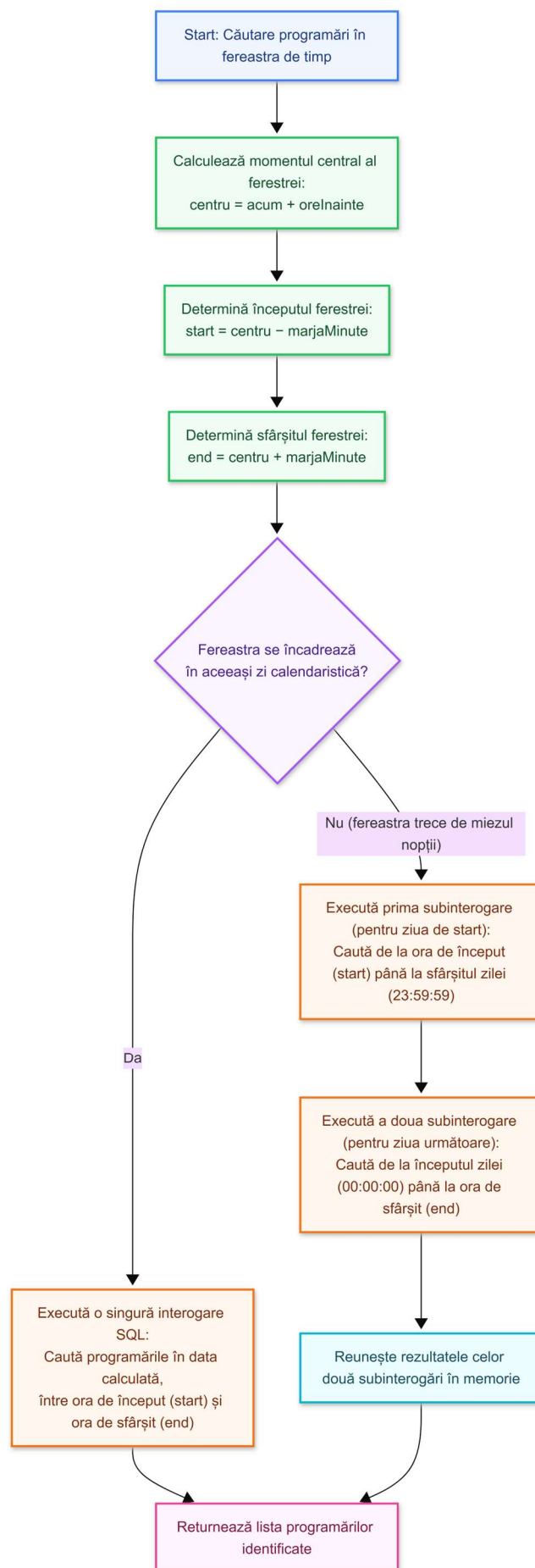


Figura A6.3. Arborele decizional pentru partiționarea temporală binară a interogărilor

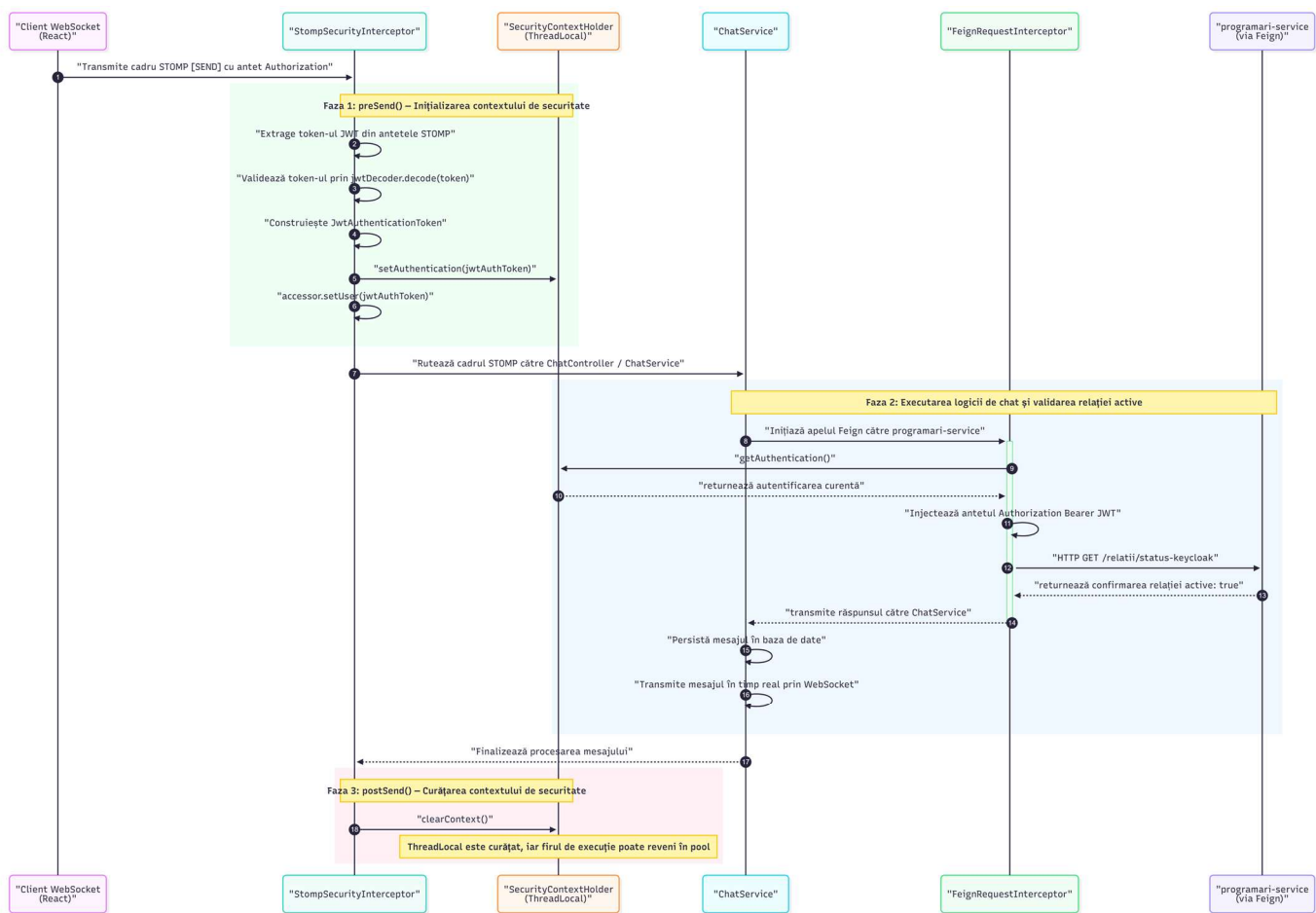


Figura A6.4. Ciclul de viață al interceptorului de securitate pe canalul STOMP

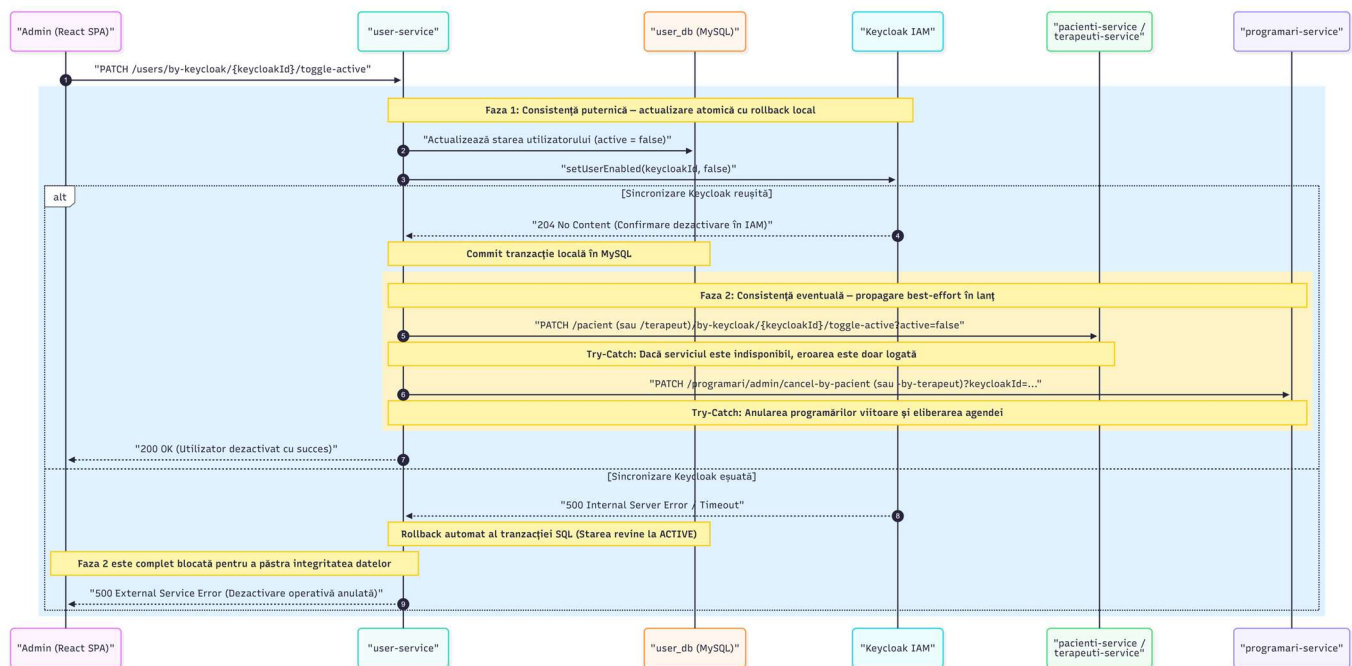


Figura A6.5. Fluxul de dezactivare a contului de utilizator cu consistență asimetrică

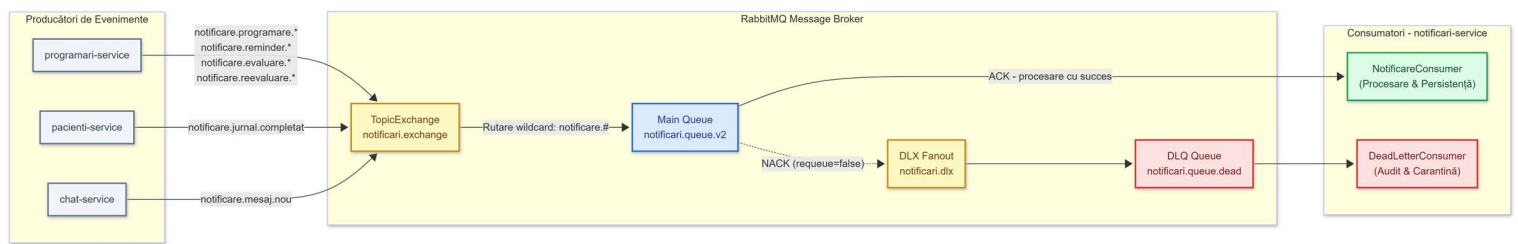


Figura A6.6. Topologia de carantinare a evenimentelor de notificare asincronă în RabbitMQ

Anexa 7. Fluxurile operaționale și interacțiunea din interfața utilizatorului

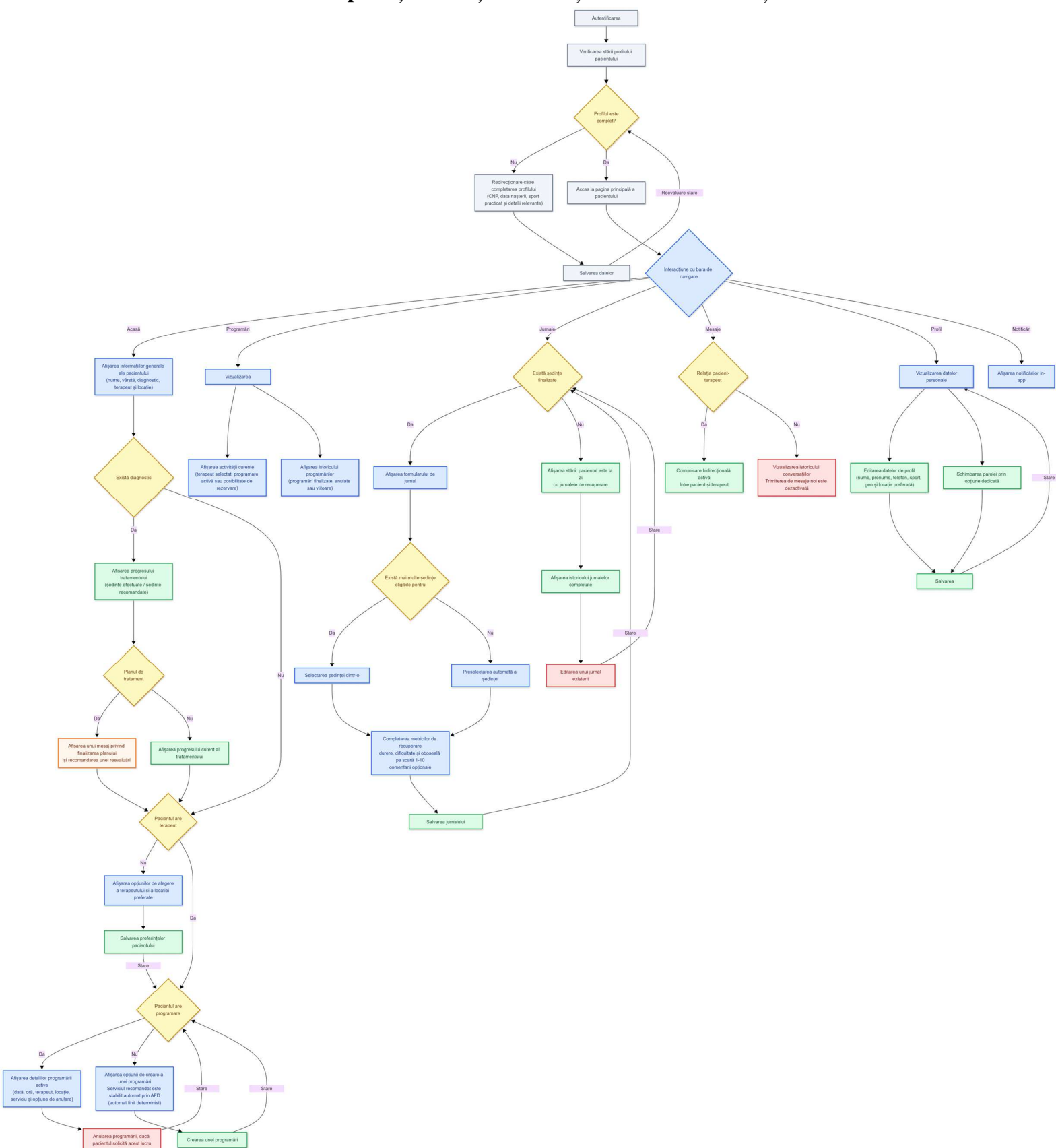


Figura A7.1. Fluxul decizional și navigarea în aplicație pentru rolul de Pacient.

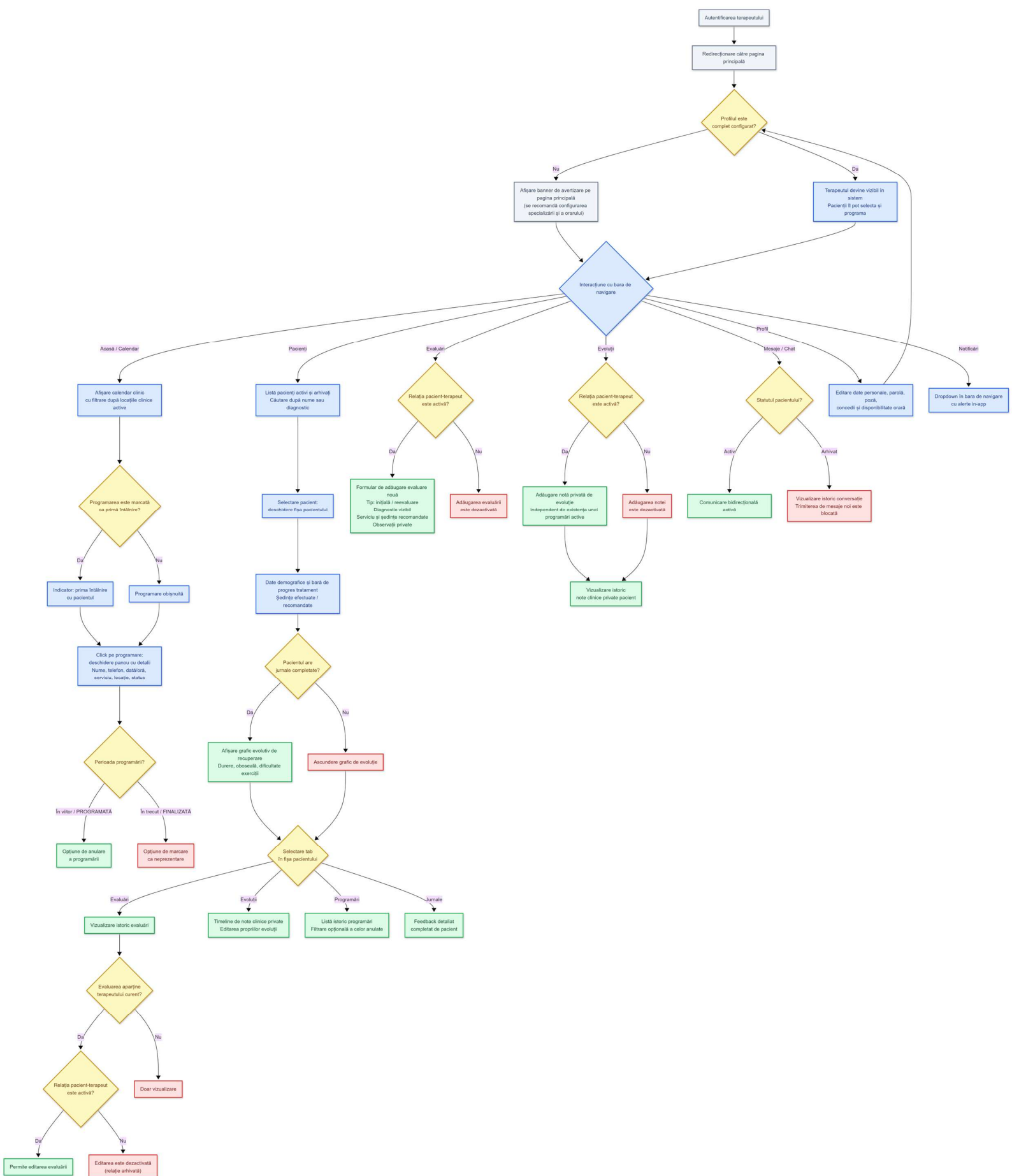


Figura A7.2. Fluxul operațional, logica de navigare și deciziile de autorizare clinică pentru rolul de Terapeut.

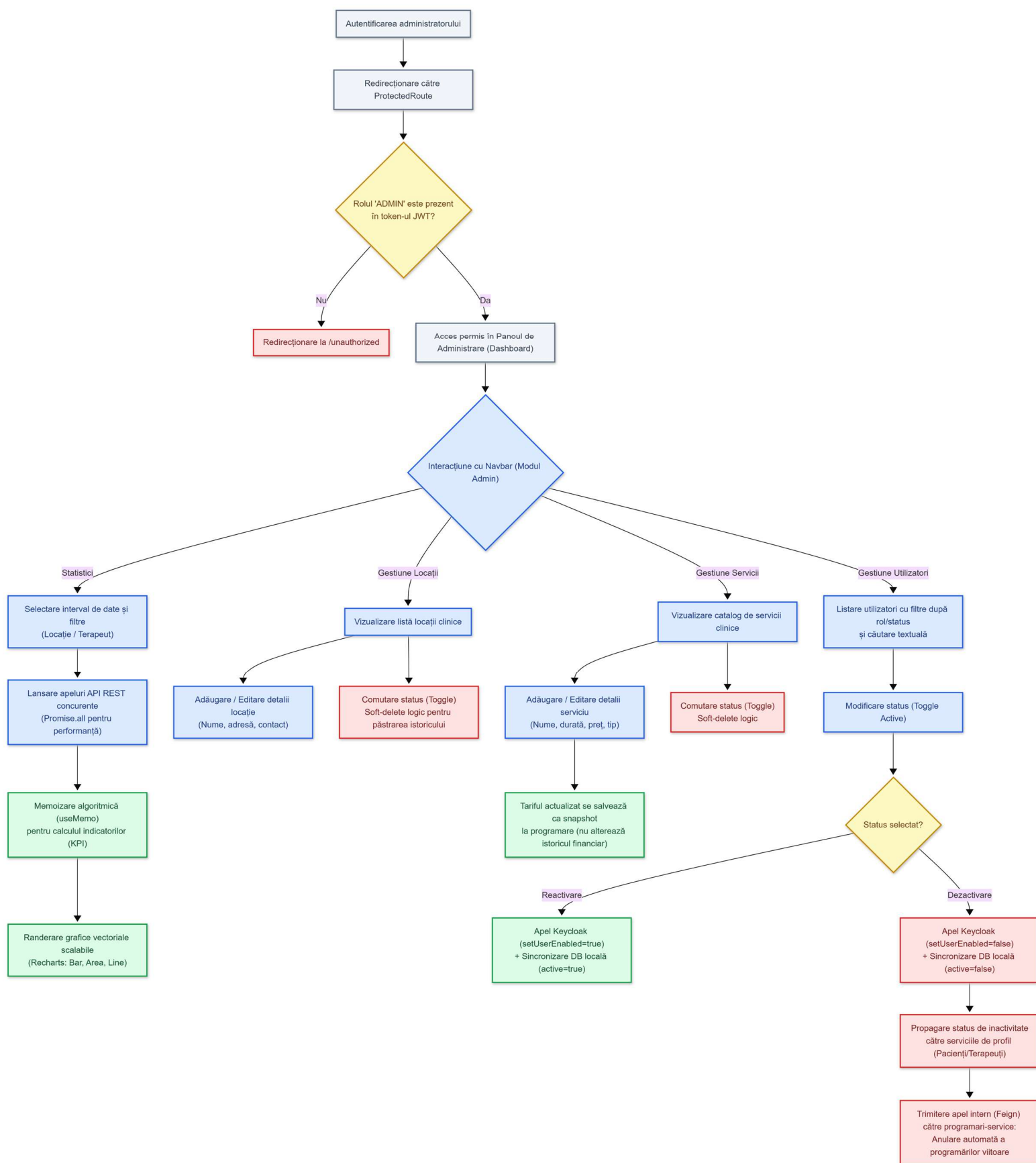


Figura A7.3. Fluxul de administrare, managementul cataloagelor și propagarea securității pentru rolul de Administrator.

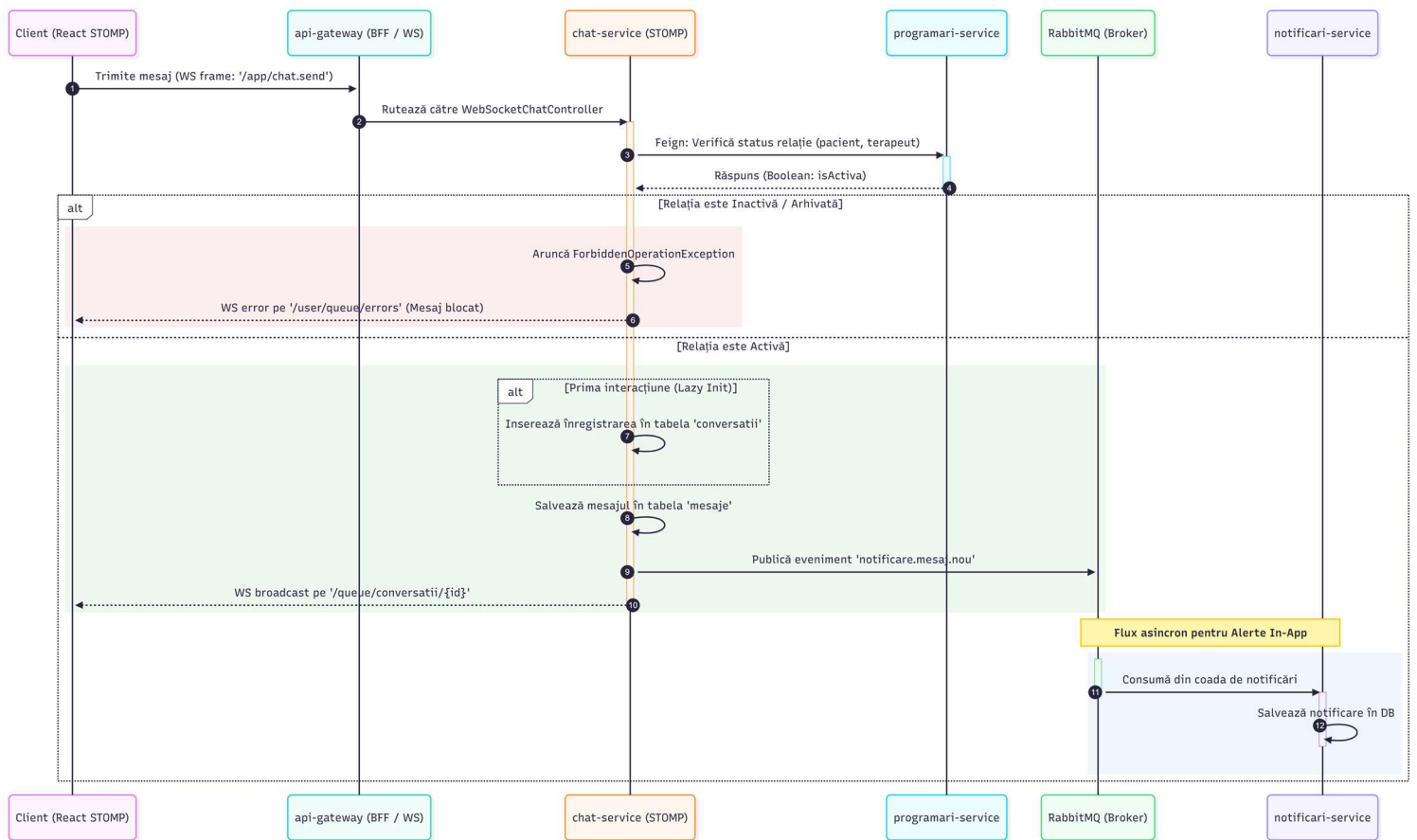
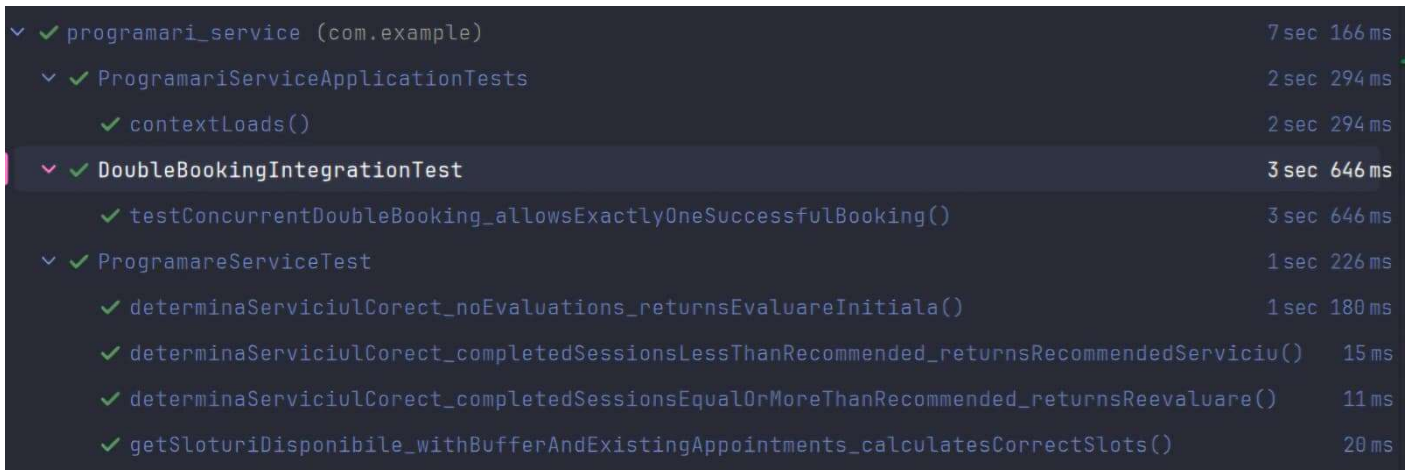


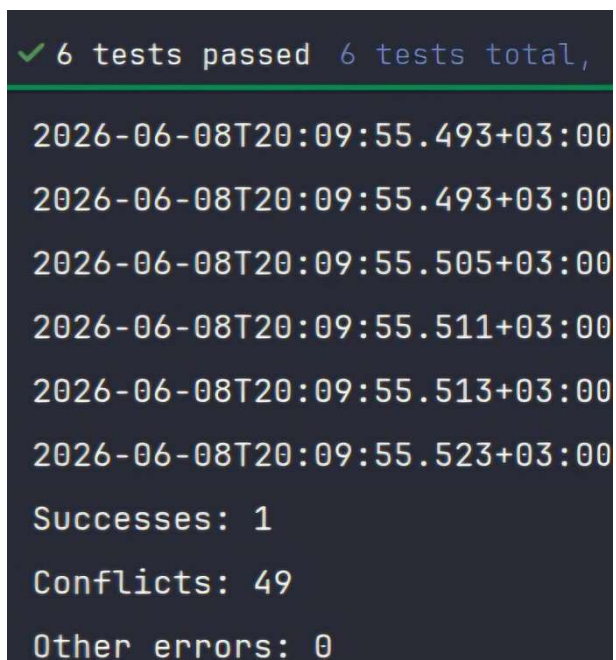
Figura A7.4. Diagrama de secvență pentru validarea mesajelor STOMP și propagarea asincronă a notificărilor.

Anexa 8. Validarea sistemului, testarea performanței și reziliența infrastructurii



✓ programari_service (com.example)	7 sec 166 ms
✓ ProgramariServiceApplicationTests	2 sec 294 ms
✓ contextLoads()	2 sec 294 ms
✓ DoubleBookingIntegrationTest	3 sec 646 ms
✓ testConcurrentDoubleBooking_allowsExactlyOneSuccessfulBooking()	3 sec 646 ms
✓ ProgramareServiceTest	1 sec 226 ms
✓ determinaServiciulCorect_noEvaluations_returnsEvaluareInitiala()	1 sec 180 ms
✓ determinaServiciulCorect_completedSessionsLessThanRecommended_returnsRecommendedServiciu()	15 ms
✓ determinaServiciulCorect_completedSessionsEqualOrMoreThanRecommended_returnsReevaluare()	11 ms
✓ getSloturiDisponibile_withBufferAndExistingAppointments_calculatesCorrectSlots()	20 ms

Figura A8.1. Rezultatele execuției testelor unitare pentru componentele algoritmice critice.



```
✓ 6 tests passed 6 tests total,  
2026-06-08T20:09:55.493+03:00  
2026-06-08T20:09:55.493+03:00  
2026-06-08T20:09:55.505+03:00  
2026-06-08T20:09:55.511+03:00  
2026-06-08T20:09:55.513+03:00  
2026-06-08T20:09:55.523+03:00  
Successes: 1  
Conflicts: 49  
Other errors: 0
```

Figura A8.2. Rezultatul testului de stres pentru validarea mecanismului de blocare pesimistă în baza de date.

RabbitMQ™ RabbitMQ 3.13.7 Erlang 26.2.5.16

Refreshed 2026-06-08 22:14:34 Refresh every 5 seconds

Virtual host All Cluster rabbit@5800d385a197 User guest Log out

Overview Connections Channels Exchanges **Queues and Streams** Admin

Queues

▼ All queues (2)

Pagination

Page 1 of 1 - Filter: ☐ Regex ?

Displaying 2 items , page size up to: 100

Overview					Messages			Message rates			
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	notificari.queue.dead	classic	D	running	1	0	1				
/	notificari.queue.v2	classic	D DLX	running	0	0	0	0.00/s	0.00/s	0.00/s	

► Add a new queue

HTTP API Documentation Tutorials New releases Commercial edition Commercial support Discussions Discord Plugins GitHub

Figura A8.3. Rutarea automată a mesajelor malformate către coada de carantinare (DLQ) în RabbitMQ.

Kinetocare

Acasă Programări Jurnal Mesaje Notificări 5 Profil

Conversații

Dumitrescu Alexandru

Ma bucur 08:59 pm

25 Martie

Buna ziua! 08:56 pm

Va multumesc pentru sedinta de azi. Ma simt mult mai bine! 08:58 pm

Ma bucur 08:59 pm

Scrie un mesaj aici...

Nu s-a putut trimite mesajul: Neautorizat! Token JWT invalid sau lipsă.

Figura A8.4. Respingerea transmiterii mesajelor pe canalul WebSocket în absența unui jeton JWT valid.



Alătură-te platformei și descoperă o experiență modernă de management al terapiei — rapidă, sigură, inteligentă.

Creează-ți contul

Completează datele de mai jos pentru a te înregistra.
Ai deja cont? [Conectează-te](#)

⚠️ Serviciul dependent este indisponibil momentan.

Name

Prenume

Gen

Feminin

Telefon

Email

Parolă

Tip cont

Pacient

Creează contul

KEYCLOAK

admin

kinetocare Current realm

Manage realms

Manage

Clients

Client scopes

Realm roles

Users

Groups

Sessions

Events

Users

Users are the users in the current realm. [Learn more](#)

User list

Default search

Q test_rollback@gmail.c...

Add user

Delete user

Refresh

0 - 0

No search results

Click on the search bar above to search again

users

Limit to 400 rows

1 • SELECT * FROM user_service.users WHERE email='test_rollback@gmail.com';

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	id	active	created_at	email	gen	keycloak_id	nume	prenume	role	telefon	created_by	last_modified_by	updated_at
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figura A8.5. Validarea mecanismului compensatoriu la înregistrare: anularea operațiunii și lipsa persistenței în sistemul de identitate (Keycloak) și în baza de date.

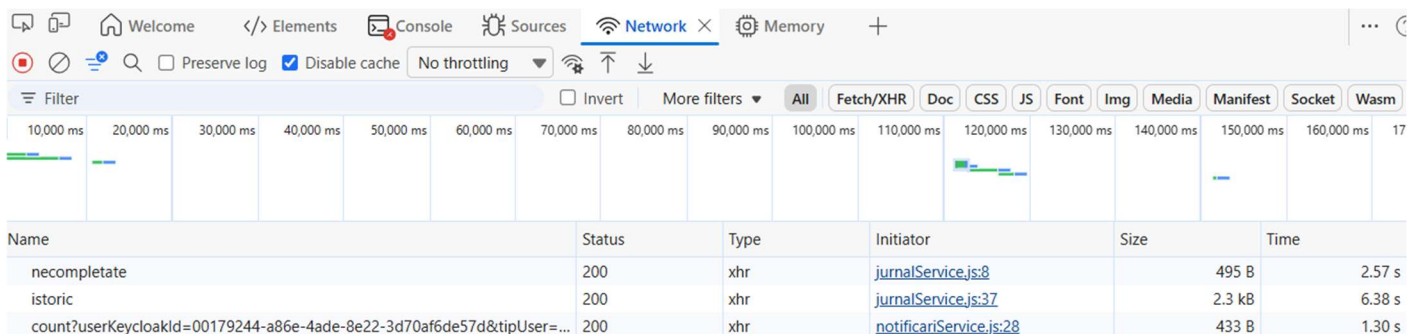


Figura A8.6. Suspendarea interogărilor redundante (Smart Polling) sincronizată cu vizibilitatea tab-ului în browser.

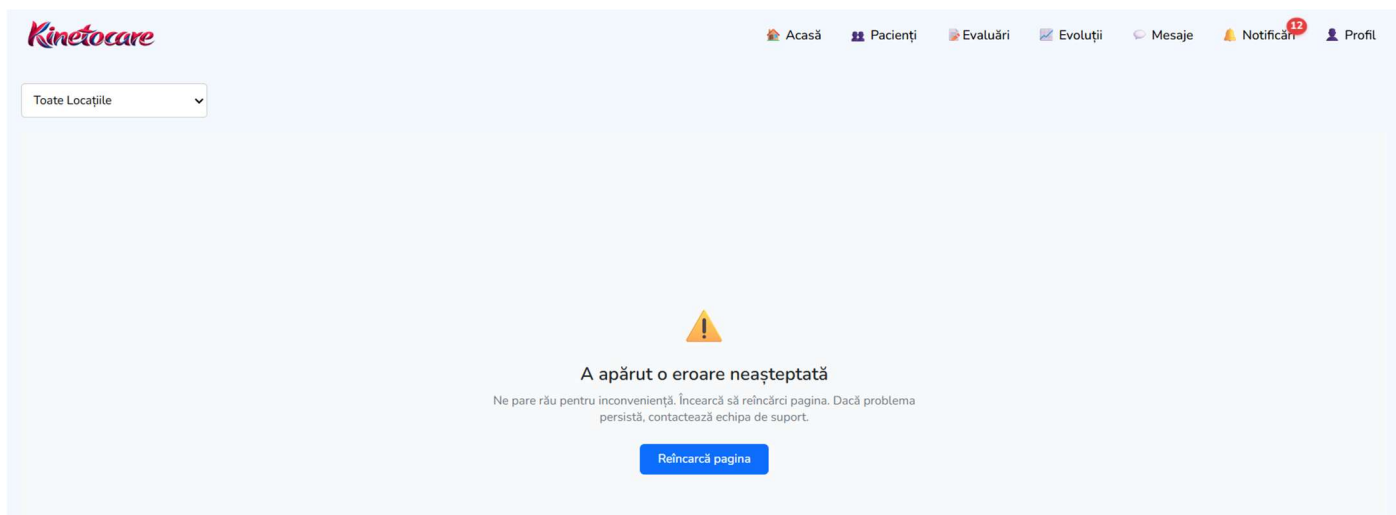


Figura A8.7. Izolarea granulară a defecțiunilor prin intermediul componentei ErrorBoundary în interfața React.

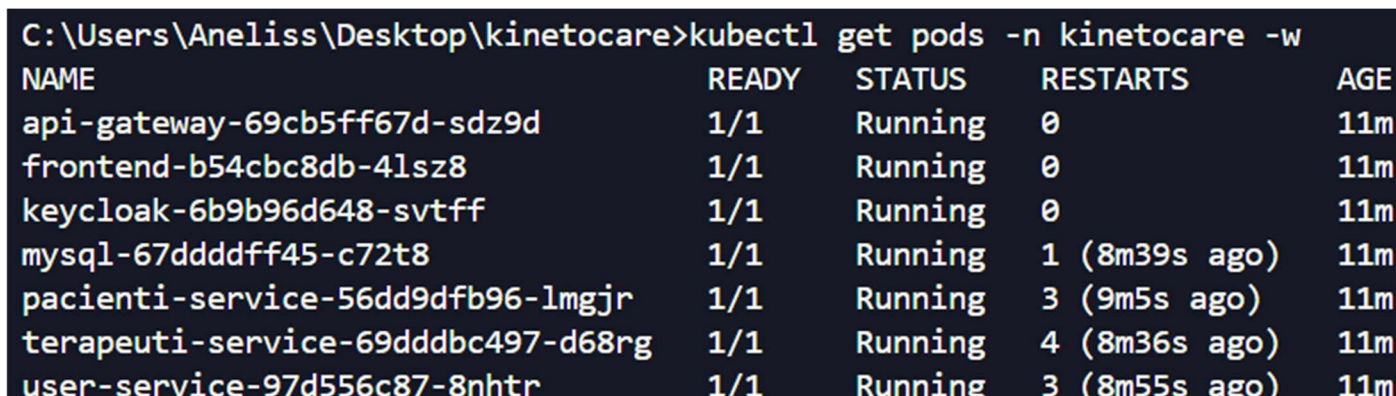


Figura A8.8. Supravegherea stării containerelor și repornirea automată a proceselor blocate în clusterul Kubernetes local.

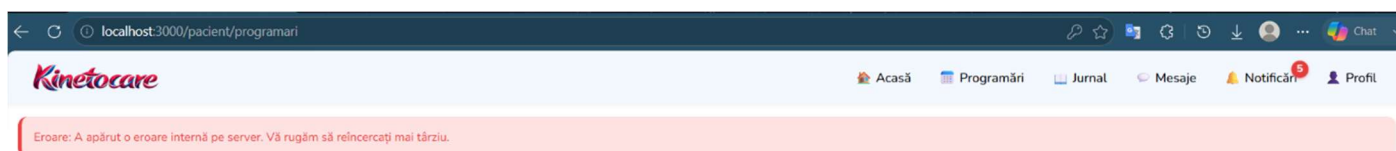


Figura A8.9. Degradarea grațioasă a interfeței grafice în cazul defecțiunii parțiale a unui microserviciu.